

Model5

May 2, 2026

1 Model 5 - Improved Budget Policy

```
[1]: # Cell 1: Install GPU-compatible torch + dependencies
# This installs the CUDA 12.8 wheel that matches the H200 NVL environment.
# After this cell finishes, do: Kernel -> Restart Kernel, then run Cell 2.
import subprocess, sys

subprocess.check_call([
    sys.executable, '-m', 'pip', 'install',
    'torch', '--index-url', 'https://download.pytorch.org/whl/cu128', '-q'
])

subprocess.check_call([
    sys.executable, '-m', 'pip', 'install',
    'numpy', 'transformers', 'datasets', 'sentence-transformers', 'accelerate',
    '-q'
])

import torch
print('torch:', torch.__version__)
print('cuda:', torch.cuda.is_available())
if torch.cuda.is_available():
    print('gpu:', torch.cuda.get_device_name(0))
print('DONE - Now restart the kernel and run Cell 2')
```

```
torch: 2.11.0+cu128
cuda: True
gpu: NVIDIA H200 NVL
DONE - Now restart the kernel and run Cell 2
```

```
[2]: # Cell 2: Full Model 5 training code
import os
import re
import json
import time
import random
import shutil
from dataclasses import dataclass
from typing import Dict, List, Tuple, Optional
```

```

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM
from sentence_transformers import SentenceTransformer

# =====
# Config
# =====

@dataclass
class Config:
    # Dataset
    train_subset: int = 300
    test_subset: int = 100

    # Models
    llm_name: str = "Qwen/Qwen2.5-1.5B-Instruct"
    embed_name: str = "sentence-transformers/all-MiniLM-L6-v2"

    # FIX 1: Oracle budgets now denser in the 160-220 region where
    # the model actually succeeds (v9 was dense around 96-160 where it fails).
    oracle_budgets: Tuple[int, ...] = (
        96, 112, 128, 144, 160, 168, 176, 184, 192, 200, 210, 220
    )

    # Supervised warm-start
    warmstart_epochs: int = 15
    warmstart_learning_rate: float = 5e-4

    # FIX 2: More RL epochs so the policy has time to evolve
    rl_epochs: int = 8
    policy_learning_rate: float = 3e-4
    value_learning_rate: float = 8e-4

    # FIX 3: Higher entropy coef prevents policy from collapsing to a
    # single fixed budget (v9 had 0.002 which was too low to maintain diversity)
    entropy_coef: float = 0.02
    value_loss_coef: float = 0.5
    grad_clip_norm: float = 1.0

    # FIX 4: Token penalty raised from 0.00005 to 0.003 so the policy

```

```

# has a real incentive to use fewer tokens when the question is easy.
# At 176 tokens: reward = 1.25 - 0.003*176 = 0.722 (correct)
# At 96 tokens: reward = 1.25 - 0.003*96 = 0.962 (correct)
# Gap = 0.24 per question vs 0.006 gap in v9 -- meaningful signal now.
reward_token_penalty: float = 0.003
use_soft_reward: bool = False
exact_match_bonus: float = 0.25

# Thinking budget range
min_budget: int = 64
max_budget: int = 220

# Final answer generation budget
answer_max_new_tokens: int = 20

# Generation
do_sample: bool = False
temperature: float = 0.0

# FIX 5: Exploration only in 128-220 where the model succeeds,
# so RL sees positive reward signals when exploring
warmup_epochs_rl: int = 3
warmup_mix_rl: float = 0.30
warmup_min_budget_high: int = 128

# FIX 6: Oracle blend in RL -- supervised loss on deterministic head
# every RL step so it doesn't freeze (root cause of v9 RL doing nothing)
use_oracle_blend_in_rl: bool = True
oracle_blend_weight: float = 0.5

# Hardware
seed: int = 42
device: str = "cuda" if torch.cuda.is_available() else "cpu"

# Logging
print_every: int = 20
debug_examples: int = 3
eval_print_every: int = 10

# Cache / outputs
cache_dir: str = "./cache_rl_v10"
clear_cache_on_start: bool = True
oracle_cache_path: str = "oracle_budget_labels_v10.json"

# Saved models
best_warmstart_policy_path: str = "best_warmstart_policy_v10.pt"

```

```

    best_warmstart_policy_by_acc_path: str = "best_warmstart_policy_by_acc_v10.
    ↪pt"
    best_policy_path: str = "best_policy_v10.pt"
    best_policy_by_acc_path: str = "best_policy_by_acc_v10.pt"
    best_value_path: str = "best_value_v10.pt"
    final_policy_path: str = "two_stage_policy_v10_final.pt"
    final_value_path: str = "two_stage_value_net_v10_final.pt"

    # Baselines
    baseline_budgets: Tuple[int, ...] = (96, 128, 144, 160, 176, 192, 220)

CFG = Config()

# =====
# Utilities
# =====

def set_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

def ensure_dir(path):
    os.makedirs(path, exist_ok=True)

def reset_cache_dir(cache_dir, retries=5, delay=0.5):
    if not os.path.exists(cache_dir):
        os.makedirs(cache_dir, exist_ok=True)
        return
    last_error = None
    for _ in range(retries):
        try:
            if os.path.exists(cache_dir):
                shutil.rmtree(cache_dir, ignore_errors=False)
            if os.path.exists(cache_dir):
                for root, dirs, files in os.walk(cache_dir, topdown=False):
                    for f in files:
                        try: os.remove(os.path.join(root, f))
                        except FileNotFoundError: pass
                    for d in dirs:
                        try: os.rmdir(os.path.join(root, d))

```

```

        except OSError: pass
        if os.path.exists(cache_dir):
            os.rmdir(cache_dir)
        break
    except OSError as e:
        last_error = e
        time.sleep(delay)
    if os.path.exists(cache_dir):
        raise RuntimeError(f"Failed to clear cache: {cache_dir}") from _
↪last_error
    os.makedirs(cache_dir, exist_ok=True)

def clamp_budget(x, min_budget, max_budget):
    return int(max(min_budget, min(max_budget, round(x))))

def denormalize_budget(x, min_budget, max_budget):
    return x * (max_budget - min_budget) + min_budget

def budget_to_normalized(budget, min_budget, max_budget):
    return float(budget - min_budget) / float(max_budget - min_budget)

def clean_model_output(text):
    if text is None:
        return ""
    cleaned = text
    for marker in ["\nHuman:", "\nAssistant:", "\nUser:", "\nSystem:",
                  "Human:", "Assistant:", "User:", "System:",
                  "You are an AI assistant", "Provide a detailed answer",
                  "I'll respond in English", "<|im_end|>", "<|endoftext|>"]:
        if marker in cleaned:
            cleaned = cleaned.split(marker)[0]
    return cleaned.strip()

# =====
# GSM8K helpers
# =====

def extract_gsm8k_final_answer(answer_text):
    if "####" in answer_text:
        return answer_text.split("####")[-1].strip()
    return answer_text.strip()

```

```

def normalize_numeric_string(text):
    if text is None:
        return ""
    text = text.strip().replace(",", "").replace("$", "").replace("%", "").
↪strip()
    matches = re.findall(r"-?\d+(?:\.\d+)?", text)
    if not matches:
        return text.lower().strip()
    return matches[-1]

def parse_final_answer(output_text):
    text = clean_model_output(output_text)
    for pattern in [
        r"FINAL ANSWER\s*:\s*(-?\d+(?:\.\d+)?)",
        r"Final Answer\s*:\s*(-?\d+(?:\.\d+)?)",
        r"Answer\s*:\s*(-?\d+(?:\.\d+)?)",
    ]:
        m = re.search(pattern, text, re.IGNORECASE)
        if m:
            return normalize_numeric_string(m.group(1))
    lines = [l.strip() for l in text.splitlines() if l.strip()]
    first_line = lines[0] if lines else text.strip()
    m = re.search(r"-?\d+(?:\.\d+)?", first_line)
    if m:
        return normalize_numeric_string(m.group(0))
    matches = re.findall(r"-?\d+(?:\.\d+)?", text)
    if matches:
        return normalize_numeric_string(matches[0])
    return normalize_numeric_string(text)

def is_correct_prediction(pred, gold):
    return int(pred == gold)

def compute_reward(pred, gold, thinking_tokens_used, cfg):
    exact = float(is_correct_prediction(pred, gold))
    # FIX 4: Meaningful token penalty -- policy now has real incentive
    # to differentiate budget per question difficulty.
    reward = exact - cfg.reward_token_penalty * float(thinking_tokens_used)
    if exact > 0.5:
        reward += cfg.exact_match_bonus
    return reward, exact

```

```

# =====
# Prompts
# =====

def build_thinking_prompt(question):
    return (
        "Solve the following grade-school math problem carefully.\n"
        "Reason step by step.\n"
        "Keep the reasoning concise and relevant.\n"
        "Do not start a conversation.\n"
        "Do not add extra instructions.\n"
        "Only produce reasoning.\n\n"
        f"Question: {question}\n\nReasoning:"
    )

def build_answer_prompt(question, thinking_text):
    return (
        "Use the reasoning below to produce the final numeric answer.\n"
        "Return only one line in this exact format:\n"
        "FINAL ANSWER: <number>\n\n"
        f"Question: {question}\n\nReasoning:\n{thinking_text}\n\nFINAL ANSWER:"
    )

# =====
# Featurizer
# =====

class QuestionFeaturizer:
    def __init__(self, embed_model_name, device):
        self.embedder = SentenceTransformer(embed_model_name, device=device)

    def handcrafted_features(self, question):
        q = question.lower()
        tokens = q.split()
        keywords = [
            "total", "left", "remaining", "each", "every",
            "more", "less", "after", "before", "twice",
            "half", "times", "altogether", "difference",
            "sum", "product", "cost", "price", "sold",
            "per", "average", "equal", "share", "then",
            "gave", "bought", "earned", "minutes", "hours",
            "days", "weeks", "fraction", "ratio", "percent",
            "another", "still", "now", "together"
        ]
        feats = np.array(

```

```

        [
            len(tokens),
            len(re.findall(r"\d+", q)),
            len(q),
            max(1, len(re.findall(r"[!?!]", q))),
            1.0 if any(w in q for w in ["more", "sum", "altogether",
↪ "total", "together"]) else 0.0,
            1.0 if any(w in q for w in ["left", "remaining", "difference",
↪ "less", "still"]) else 0.0,
            1.0 if any(w in q for w in ["each", "every", "times",
↪ "product", "twice"]) else 0.0,
            1.0 if any(w in q for w in ["per", "average", "equally",
↪ "half", "share", "ratio"]) else 0.0,
            1.0 if any(w in q for w in ["then", "after", "before",
↪ "altogether", "remaining", "now"]) else 0.0,
            1.0 if any(w in q for w in ["minutes", "hours", "days",
↪ "weeks"]) else 0.0,
            1.0 if "$" in q or any(w in q for w in ["dollar", "dollars",
↪ "cost", "price"]) else 0.0,
            1.0 if any(w in q for w in ["half", "fraction", "percent",
↪ "ratio"]) else 0.0,
        ] + [1.0 if kw in q else 0.0 for kw in keywords],
        dtype=np.float32
    )
    return feats

    def encode(self, question):
        emb = self.embedder.encode(
            question, convert_to_numpy=True, normalize_embeddings=True
        ).astype(np.float32)
        return np.concatenate([emb, self.handcrafted_features(question)],
↪ axis=0)

# =====
# Policy + Value Networks
# =====

class ContinuousBudgetPolicy(nn.Module):
    def __init__(self, input_dim, hidden_dim=256):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim), nn.ReLU(),
        )
        self.mean_head = nn.Linear(hidden_dim, 1)

```



```

        # FIX 6: Start log_std at 0.5 (std=1.65) for stronger initial
        ↪ exploration
        # v9 used 0.0 which collapsed the distribution too quickly
        self.log_std = nn.Parameter(torch.tensor([0.5], dtype=torch.float32))

    def forward(self, x):
        h = self.net(x)
        mean = self.mean_head(h)
        return mean, self.log_std.expand_as(mean)

    def sample_budget(self, x):
        mean, log_std = self.forward(x)
        dist = torch.distributions.Normal(mean, torch.exp(log_std))
        raw = dist.rsample()
        normalized = torch.sigmoid(raw)
        return normalized, dist.log_prob(raw).sum(dim=-1), dist.entropy().
        ↪ sum(dim=-1)

    def deterministic_budget(self, x):
        mean, _ = self.forward(x)
        return torch.sigmoid(mean)

class ValueNetwork(nn.Module):
    def __init__(self, input_dim, hidden_dim=256):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, 1),
        )

    def forward(self, x):
        return self.net(x).squeeze(-1)

# =====
# LLM wrapper
# =====

class LLMReasoner:
    def __init__(self, model_name, device):
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        if self.tokenizer.pad_token is None:
            self.tokenizer.pad_token = self.tokenizer.eos_token
        dtype = torch.float16 if device == "cuda" else torch.float32
        self.model = AutoModelForCausalLM.from_pretrained(

```

```

        model_name, torch_dtype=dtype, device_map="auto"
    )
    self.model.eval()
    self.device = device
    if hasattr(self.model, "generation_config") and self.model.
↪generation_config is not None:
        self.model.generation_config.do_sample = False
        self.model.generation_config.temperature = None
        self.model.generation_config.top_p = None
        self.model.generation_config.top_k = None

    def generate(self, prompt, max_new_tokens, do_sample=False, temperature=0.
↪0):
        inputs = self.tokenizer(prompt, return_tensors="pt").to(self.device)
        input_len = inputs["input_ids"].shape[1]
        gen_kwargs = {
            "max_new_tokens": max_new_tokens,
            "do_sample": do_sample,
            "pad_token_id": self.tokenizer.eos_token_id,
            "eos_token_id": self.tokenizer.eos_token_id,
        }
        if do_sample:
            gen_kwargs["temperature"] = temperature
        with torch.no_grad():
            outputs = self.model.generate(**inputs, **gen_kwargs)
            gen_ids = outputs[0][input_len:]
            text = clean_model_output(self.tokenizer.decode(gen_ids,
↪skip_special_tokens=True))
            return {"text": text, "tokens_used": len(gen_ids)}

# =====
# Caching
# =====

def cache_key(stage, question, extra, model_name):
    import hashlib
    return hashlib.md5(f"{stage}|||{model_name}|||{question}|||{extra}".
↪encode()).hexdigest()

def load_from_cache(cache_dir, key):
    path = os.path.join(cache_dir, f"{key}.json")
    if os.path.exists(path):
        with open(path, "r", encoding="utf-8") as f:
            return json.load(f)
    return None

```

```

def save_to_cache(cache_dir, key, value):
    path = os.path.join(cache_dir, f"{key}.json")
    with open(path, "w", encoding="utf-8") as f:
        json.dump(value, f, ensure_ascii=False, indent=2)

# =====
# Two-stage example run
# =====

def run_two_stage_example(question, gold_answer, thinking_budget, llm, cfg):
    ensure_dir(cfg.cache_dir)

    think_key = cache_key("thinking", question, f"budget={thinking_budget}",
        ↪cfg.llm_name)
    cached = load_from_cache(cfg.cache_dir, think_key)
    if cached:
        thinking_text = clean_model_output(cached["text"])
        thinking_tokens_used = cached["tokens_used"]
    else:
        r = llm.generate(build_thinking_prompt(question), thinking_budget,
            cfg.do_sample, cfg.temperature)
        thinking_text = clean_model_output(r["text"])
        thinking_tokens_used = r["tokens_used"]
        save_to_cache(cfg.cache_dir, think_key, {"text": thinking_text,
            ↪"tokens_used": thinking_tokens_used})

    answer_key = cache_key("answer", question, thinking_text, cfg.llm_name)
    cached = load_from_cache(cfg.cache_dir, answer_key)
    if cached:
        answer_text = clean_model_output(cached["text"])
        answer_tokens_used = cached["tokens_used"]
    else:
        r = llm.generate(build_answer_prompt(question, thinking_text),
            cfg.answer_max_new_tokens, False, 0.0)
        answer_text = clean_model_output(r["text"])
        answer_tokens_used = r["tokens_used"]
        save_to_cache(cfg.cache_dir, answer_key, {"text": answer_text,
            ↪"tokens_used": answer_tokens_used})

    pred = parse_final_answer(answer_text)
    reward, exact = compute_reward(pred, gold_answer, thinking_tokens_used, cfg)
    return {
        "prediction": pred, "gold": gold_answer, "correct": int(exact),

```

```

        "thinking_tokens_used": thinking_tokens_used, "answer_tokens_used":␣
↪answer_tokens_used,
        "reward": reward, "thinking_text": thinking_text, "answer_text":␣
↪answer_text,
        "thinking_budget": thinking_budget,
    }

# =====
# Debug
# =====

def print_debug_examples(data, llm, cfg, budget, n=3):
    print("\n" + "=" * 100)
    print(f"DEBUG EXAMPLES WITH FIXED THINKING BUDGET = {budget}")
    print("=" * 100)
    for i, item in enumerate(data[:n]):
        question = item["question"]
        gold =␣
↪normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
        result = run_two_stage_example(question, gold, budget, llm, cfg)
        print(f"\n[Example {i+1}]\n" + "-" * 100)
        print(f"QUESTION: {question}")
        print(f"\nTHINKING TEXT:\n{result['thinking_text']}")
        print(f"\nANSWER TEXT:\n{result['answer_text']}")
        print(f"\nPARSED PREDICTION: {result['prediction']}")
        print(f"GOLD ANSWER      : {gold}")
        print(f"CORRECT              : {result['correct']}")
        print(f"THINK TOKENS         : {result['thinking_tokens_used']}")
        print(f"REWARD               : {result['reward']}")
        print("-" * 100)

# =====
# Evaluation
# =====

def evaluate_fixed_budget(data, llm, cfg, fixed_budget):
    corrects, tokens, rewards = [], [], []
    print(f"\nEvaluating fixed budget = {fixed_budget} on {len(data)} examples..
↪.")
    for idx, item in enumerate(data):
        q = item["question"]
        gold =␣
↪normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
        r = run_two_stage_example(q, gold, fixed_budget, llm, cfg)

```

```

        corrects.append(r["correct"]); tokens.append(r["thinking_tokens_used"]);
    ↪ rewards.append(r["reward"])
        if (idx + 1) % cfg.eval_print_every == 0 or (idx + 1) == len(data):
            print(f" Fixed {fixed_budget} | {idx+1}/{len(data)} | Acc={np.
    ↪mean(corrects):.4f} | "
                    f"AvgTokens={np.mean(tokens):.1f} | AvgReward={np.
    ↪mean(rewards):.4f}")
        return {"accuracy": float(np.mean(corrects)), "avg_thinking_tokens":
    ↪float(np.mean(tokens)),
                "avg_reward": float(np.mean(rewards)), "budget": fixed_budget}

def evaluate_policy_deterministic(data, featurizer, policy, llm, cfg):
    policy.eval()
    corrects, tokens, rewards, budgets = [], [], [], []
    with torch.no_grad():
        for idx, item in enumerate(data):
            q = item["question"]
            gold =
    ↪normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
            x = torch.tensor(featurizer.encode(q), dtype=torch.float32,
    ↪device=cfg.device).unsqueeze(0)
            budget = clamp_budget(denormalize_budget(policy.
    ↪deterministic_budget(x).item(),
                                cfg.min_budget, cfg.
    ↪max_budget),
                                cfg.min_budget, cfg.max_budget)
            r = run_two_stage_example(q, gold, budget, llm, cfg)
            corrects.append(r["correct"]); tokens.
    ↪append(r["thinking_tokens_used"])
            rewards.append(r["reward"]); budgets.append(budget)
            if (idx + 1) % cfg.eval_print_every == 0 or (idx + 1) == len(data):
                print(f" Det eval | {idx+1}/{len(data)} | Acc={np.
    ↪mean(corrects):.4f} | "
                        f"AvgBudget={np.mean(budgets):.2f} | StdBudget={np.
    ↪std(budgets):.2f}")
            return {"accuracy": float(np.mean(corrects)), "avg_thinking_tokens":
    ↪float(np.mean(tokens)),
                    "avg_reward": float(np.mean(rewards)), "avg_budget": float(np.
    ↪mean(budgets)),
                    "min_budget": float(np.min(budgets)), "max_budget": float(np.
    ↪max(budgets)),
                    "std_budget": float(np.std(budgets))}

def evaluate_policy_sampled(data, featurizer, policy, llm, cfg):

```

```

policy.eval()
corrects, tokens, rewards, budgets = [], [], [], []
with torch.no_grad():
    for idx, item in enumerate(data):
        q = item["question"]
        gold = _
        ↪normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
        x = torch.tensor(featurizer.encode(q), dtype=torch.float32, _
        ↪device=cfg.device).unsqueeze(0)
        norm_budget, _, _ = policy.sample_budget(x)
        budget = clamp_budget(denormalize_budget(norm_budget.item(), cfg.
        ↪min_budget, cfg.max_budget),
                                cfg.min_budget, cfg.max_budget)
        r = run_two_stage_example(q, gold, budget, llm, cfg)
        corrects.append(r["correct"]); tokens.
        ↪append(r["thinking_tokens_used"])
        rewards.append(r["reward"]); budgets.append(budget)
        if (idx + 1) % cfg.eval_print_every == 0 or (idx + 1) == len(data):
            print(f" Samp eval | {idx+1}/{len(data)} | Acc={np.
        ↪mean(corrects):.4f} | "
                    f"AvgBudget={np.mean(budgets):.2f}")
        return {"accuracy": float(np.mean(corrects)), "avg_thinking_tokens":_
        ↪float(np.mean(tokens)),
                "avg_reward": float(np.mean(rewards)), "avg_budget": float(np.
        ↪mean(budgets)),
                "min_budget": float(np.min(budgets)), "max_budget": float(np.
        ↪max(budgets)),
                "std_budget": float(np.std(budgets))}

# =====
# Oracle budget construction
# FIX 1+2: Select MINIMUM budget that achieves correctness, not
# the one with highest raw reward. This teaches efficiency.
# =====

def build_oracle_budget_labels(train_data, llm, cfg):
    if os.path.exists(cfg.oracle_cache_path):
        print(f"\nLoading oracle labels from {cfg.oracle_cache_path} ...")
        with open(cfg.oracle_cache_path, "r", encoding="utf-8") as f:
            return json.load(f)

    print("\nBuilding oracle budget labels...")
    oracle_records = []

    for idx, item in enumerate(train_data):

```

```

        question = item["question"]
        gold = _
        ↪normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
        budget_rewards = {}

        for budget in cfg.oracle_budgets:
            result = run_two_stage_example(question, gold, budget, llm, cfg)
            budget_rewards[str(budget)] = {
                "reward": float(result["reward"]),
                "correct": int(result["correct"]),
                "thinking_tokens_used": int(result["thinking_tokens_used"]),
                "prediction": result["prediction"],
            }

            # Pick the MINIMUM budget that gets the right answer.
            # This directly teaches the policy to be efficient.
            correct_budgets = [int(b) for b, v in budget_rewards.items() if _
        ↪v["correct"] == 1]
            if correct_budgets:
                best_budget = min(correct_budgets)
            else:
                best_budget = int(max(budget_rewards, key=lambda b: _
        ↪budget_rewards[b]["reward"]))

            best_reward = budget_rewards[str(best_budget)]["reward"]
            oracle_records.append({
                "question": question,
                "gold_answer": gold,
                "best_budget": best_budget,
                "best_reward": float(best_reward),
                "budget_rewards": budget_rewards,
                "any_correct": len(correct_budgets) > 0,
            })

            if (idx + 1) % cfg.eval_print_every == 0 or (idx + 1) == _
        ↪len(train_data):
                frac_solvable = np.mean([r["any_correct"] for r in oracle_records])
                print(f" Oracle | {idx+1}/{len(train_data)} | "
                    f"AvgBestBudget={np.mean([r['best_budget'] for r in _
        ↪oracle_records]):.1f} | "
                    f"AvgBestReward={np.mean([r['best_reward'] for r in _
        ↪oracle_records]):.4f} | "
                    f"FracSolvable={frac_solvable:.3f}")

            with open(cfg.oracle_cache_path, "w", encoding="utf-8") as f:
                json.dump(oracle_records, f, ensure_ascii=False, indent=2)

```

```

print(f"Saved oracle labels to {cfg.oracle_cache_path}")
return oracle_records

# =====
# Supervised warm-start
# FIX 3: Weight by oracle budget deviation from mean, not just reward.
# Questions where budget matters most get higher loss weight.
# =====

def warmstart_policy_from_oracle(oracle_records, test_data, featurizer, policy, llm, cfg):
    print("\nStarting supervised warm-start from oracle budgets...")
    optimizer = optim.Adam(policy.parameters(), lr=cfg.warmstart_learning_rate)
    mse_loss = nn.MSELoss(reduction="none")

    all_budgets = np.array([r["best_budget"] for r in oracle_records], dtype=np.float32)
    budget_mean = float(all_budgets.mean())
    budget_std = float(all_budgets.std()) + 1e-8

    best_eval_reward = -1e9
    best_eval_accuracy = -1.0

    for epoch in range(cfg.warmstart_epochs):
        policy.train()
        random.shuffle(oracle_records)
        losses, pred_budgets, target_budgets = [], [], []

        for step_idx, record in enumerate(oracle_records):
            target_budget = int(record["best_budget"])
            x = torch.tensor(featurizer.encode(record["question"]),
                             dtype=torch.float32, device=cfg.device).unsqueeze(0)
            pred_norm = policy.deterministic_budget(x)
            target_norm = torch.tensor(
                [[budget_to_normalized(target_budget, cfg.min_budget, cfg.max_budget)]],
                dtype=torch.float32, device=cfg.device)

            # Combined weight: reward-based (only learn from solvable cases)
            # + deviation-based (emphasize questions where budget matters)
            reward_weight = max(0.1, float(record["best_reward"]))
            deviation_weight = 1.0 + abs(target_budget - budget_mean) / budget_std

            loss = reward_weight * deviation_weight * mse_loss(pred_norm, target_norm).mean()

```



```

optimizer.zero_grad()
loss.backward()
torch.nn.utils.clip_grad_norm_(policy.parameters(), cfg.
↪grad_clip_norm)
optimizer.step()

losses.append(float(loss.item()))
pred_budget = clamp_budget(denormalize_budget(pred_norm.item(),
                                                cfg.min_budget, cfg.max_budget),
                           cfg.min_budget, cfg.max_budget)
pred_budgets.append(pred_budget)
target_budgets.append(target_budget)

if (step_idx + 1) % cfg.print_every == 0 or (step_idx + 1) ==
↪len(oracle_records):
    mae = np.mean(np.abs(np.array(pred_budgets) - np.
↪array(target_budgets)))
    print(f"[Warmstart {epoch+1}/{cfg.warmstart_epochs}] Step
↪{step_idx+1}/{len(oracle_records)} | "
          f"Loss={np.mean(losses):.6f} | PredAvg={np.
↪mean(pred_budgets):.1f} | "
          f"TargetAvg={np.mean(target_budgets):.1f} | MAE={mae:.
↪2f}")

    print(f"Warmstart epoch {epoch+1} summary | Loss={np.mean(losses):.6f}
↪| "
          f"PredAvg={np.mean(pred_budgets):.1f} | TargetAvg={np.
↪mean(target_budgets):.1f}")

    print("\nWarm-start deterministic evaluation on test set...")
    eval_det = evaluate_policy_deterministic(test_data, featurizer, policy,
↪llm, cfg)

    if eval_det["avg_reward"] > best_eval_reward:
        best_eval_reward = eval_det["avg_reward"]
        torch.save(policy.state_dict(), cfg.best_warmstart_policy_path)
        print(f"Saved best warm-start policy by reward = {best_eval_reward:.
↪4f}\n")

    if eval_det["accuracy"] > best_eval_accuracy:
        best_eval_accuracy = eval_det["accuracy"]
        torch.save(policy.state_dict(), cfg.
↪best_warmstart_policy_by_acc_path)
        print(f"Saved best warm-start policy by accuracy =
↪{best_eval_accuracy:.4f}\n")

```

```

# =====
# RL fine-tuning
# FIX 6: Oracle blend -- supervised loss on deterministic head every
# RL step so it never freezes. This was the core bug in v9 where
# test accuracy was identical across all 5 RL epochs.
# =====

def maybe_mix_with_uniform_budget(budget, cfg, epoch_idx):
    if epoch_idx >= cfg.warmup_epochs_rl:
        return budget
    if random.random() < cfg.warmup_mix_rl:
        return random.randint(cfg.warmup_min_budget_high, cfg.max_budget)
    return budget

def train_policy_with_value(train_data, test_data, featurizer, policy,
    ↪value_net,
                               llm, cfg, oracle_records=None):
    print("\nStarting RL fine-tuning...")

    oracle_lookup = {}
    if cfg.use_oracle_blend_in_rl and oracle_records is not None:
        for rec in oracle_records:
            oracle_lookup[rec["question"]] = rec["best_budget"]

    policy_optimizer = optim.Adam(policy.parameters(), lr=cfg.
    ↪policy_learning_rate)
    value_optimizer = optim.Adam(value_net.parameters(), lr=cfg.
    ↪value_learning_rate)
    mse_loss_fn = nn.MSELoss()

    best_test_reward = -1e9
    best_test_accuracy = -1.0

    for epoch in range(cfg.rl_epochs):
        policy.train()
        value_net.train()
        random.shuffle(train_data)

        epoch_rewards, epoch_corrects, epoch_tokens, epoch_budgets = [], [],
    ↪[], []
        recent = []

        for step_idx, item in enumerate(train_data):
            question = item["question"]

```

```

        gold = ␣
        ↪normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
        x = torch.tensor(featurizer.encode(question), dtype=torch.float32, ␣
        ↪device=cfg.device).unsqueeze(0)

        norm_budget, log_prob, entropy = policy.sample_budget(x)
        sampled_budget = clamp_budget(
            denormalize_budget(norm_budget.item(), cfg.min_budget, cfg.
        ↪max_budget),
            cfg.min_budget, cfg.max_budget)
        budget = maybe_mix_with_uniform_budget(sampled_budget, cfg, epoch)

        result = run_two_stage_example(question, gold, budget, llm, cfg)
        reward = float(result["reward"])
        reward_tensor = torch.tensor([reward], dtype=torch.float32, ␣
        ↪device=cfg.device)

        value_pred = value_net(x)
        advantage = reward_tensor - value_pred.detach()

        # Core RL loss
        rl_loss = -(log_prob * advantage + cfg.entropy_coef * entropy).
        ↪mean()

        # FIX 6: Oracle blend -- supervised MSE on deterministic head
        # This is the fix for the RL-does-nothing bug in v9.
        # The deterministic head now receives a gradient every single step.
        supervised_loss = torch.tensor(0.0, device=cfg.device)
        if cfg.use_oracle_blend_in_rl and question in oracle_lookup:
            oracle_budget = oracle_lookup[question]
            oracle_norm = torch.tensor(
                [[budget_to_normalized(oracle_budget, cfg.min_budget, cfg.
        ↪max_budget)]],
                dtype=torch.float32, device=cfg.device)
            supervised_loss = mse_loss_fn(policy.deterministic_budget(x), ␣
        ↪oracle_norm)

        total_policy_loss = rl_loss + cfg.oracle_blend_weight * ␣
        ↪supervised_loss

        policy_optimizer.zero_grad()
        total_policy_loss.backward()
        torch.nn.utils.clip_grad_norm_(policy.parameters(), cfg.
        ↪grad_clip_norm)
        policy_optimizer.step()

```

```

        value_optimizer.zero_grad()
        (cfg.value_loss_coef * mse_loss_fn(value_pred, reward_tensor)).
↪backward()
        torch.nn.utils.clip_grad_norm_(value_net.parameters(), cfg.
↪grad_clip_norm)
        value_optimizer.step()

        epoch_rewards.append(reward); epoch_corrects.
↪append(result["correct"])
        epoch_tokens.append(result["thinking_tokens_used"]); epoch_budgets.
↪append(budget)
        recent.append(budget)
        if len(recent) > 10: recent.pop(0)

        if (step_idx + 1) % cfg.print_every == 0 or (step_idx + 1) ==
↪len(train_data):
            print(f"RL {epoch+1}/{cfg.rl_epochs} Step {step_idx+1}/
↪{len(train_data)} | "
                  f"Reward={np.mean(epoch_rewards):.4f} | Acc={np.
↪mean(epoch_corrects):.4f} | "
                  f"AvgBudget={np.mean(epoch_budgets):.1f} | StdBudget={np.
↪std(epoch_budgets):.1f}")
            print(f"  Recent budgets: {recent}")

        print("\n" + "=" * 90)
        print(f"RL Epoch {epoch+1} summary")
        print(f"Train Accuracy : {np.mean(epoch_corrects):.4f}")
        print(f"Train Avg Tokens: {np.mean(epoch_tokens):.1f}")
        print(f"Train Avg Reward: {np.mean(epoch_rewards):.4f}")
        print(f"Train Avg Budget: {np.mean(epoch_budgets):.1f} +/- {np.
↪std(epoch_budgets):.1f}")

        print("\nDeterministic policy evaluation on test set...")
        eval_det = evaluate_policy_deterministic(test_data, featurizer, policy,
↪llm, cfg)
        print(f"Test Accuracy   : {eval_det['accuracy']:.4f}")
        print(f"Test AvgTokens   : {eval_det['avg_thinking_tokens']:.1f}")
        print(f"Test AvgReward   : {eval_det['avg_reward']:.4f}")
        print(f"Test Budget     : {eval_det['avg_budget']:.1f} +/-
↪{eval_det['std_budget']:.1f} "
              f"[{eval_det['min_budget']:.0f}, {eval_det['max_budget']:.0f}]")
        print("=" * 90 + "\n")

        if eval_det["avg_reward"] > best_test_reward:
            best_test_reward = eval_det["avg_reward"]
            torch.save(policy.state_dict(), cfg.best_policy_path)

```

```

        torch.save(value_net.state_dict(), cfg.best_value_path)
        print(f"Saved best RL models by reward = {best_test_reward:.4f}\n")

    if eval_det["accuracy"] > best_test_accuracy:
        best_test_accuracy = eval_det["accuracy"]
        torch.save(policy.state_dict(), cfg.best_policy_by_acc_path)
        print(f"Saved best RL policy by accuracy = {best_test_accuracy:.
↪4f}\n")

# =====
# Main
# =====

def main():
    set_seed(CFG.seed)

    print("torch version:", torch.__version__)
    print("cuda available:", torch.cuda.is_available())
    print("cuda device count:", torch.cuda.device_count())
    if torch.cuda.is_available():
        print("gpu name:", torch.cuda.get_device_name(0))
    else:
        print("WARNING: Running on CPU -- this will be very slow!")

    if CFG.clear_cache_on_start:
        print(f"\nClearing old cache at: {CFG.cache_dir}")
        reset_cache_dir(CFG.cache_dir)
    else:
        ensure_dir(CFG.cache_dir)

    print(f"Using device: {CFG.device}")
    print("Loading GSM8K...")
    ds = load_dataset("gsm8k", "main")
    train_data = list(ds["train"].select(range(min(CFG.train_subset,
↪len(ds["train"]))))))
    test_data = list(ds["test"].select(range(min(CFG.test_subset,
↪len(ds["test"]))))))
    print(f"Train subset: {len(train_data)} | Test subset: {len(test_data)}")

    print("Loading featurizer...")
    featurizer = QuestionFeaturizer(CFG.embed_name, CFG.device)
    input_dim = featurizer.encode(train_data[0]["question"]).shape[0]

    print("Loading policy and value network...")
    policy = ContinuousBudgetPolicy(input_dim=input_dim).to(CFG.device)
    value_net = ValueNetwork(input_dim=input_dim).to(CFG.device)

```

```

    # Initialize mean_head bias toward the high-accuracy region (~176 tokens,
    ↪norm ~0.72)
    with torch.no_grad():
        policy.mean_head.bias.fill_(0.85)

    print("Loading LLM...")
    llm = LLMReasoner(CFG.llm_name, CFG.device)

    print_debug_examples(train_data, llm, CFG, budget=176, n=CFG.debug_examples)

    print("\n" + "=" * 80)
    print("FIXED BUDGET BASELINES ON TEST SET")
    print("=" * 80)
    baseline_results = {}
    for b in CFG.baseline_budgets:
        res = evaluate_fixed_budget(test_data, llm, CFG, b)
        baseline_results[b] = res
        print(f"Fixed {b:>3} | Acc={res['accuracy']:.4f} | "
              f"AvgTokens={res['avg_thinking_tokens']:.0f} |
    ↪AvgReward={res['avg_reward']:.4f}")

    oracle_records = build_oracle_budget_labels(train_data, llm, CFG)

    # Print oracle distribution to verify FIX 1 is working
    best_budgets = [r["best_budget"] for r in oracle_records]
    print(f"\nOracle stats: mean={np.mean(best_budgets):.1f}, std={np.
    ↪std(best_budgets):.1f}, "
          f"solvable={np.mean([r['any_correct'] for r in oracle_records]):.3f}")
    for b in sorted(set(best_budgets)):
        c = sum(1 for x in best_budgets if x == b)
        print(f"  budget={b}: {c} questions ({100*c/len(best_budgets):.1f}%)")

    warmstart_policy_from_oracle(oracle_records, test_data, featurizer, policy,
    ↪llm, cfg=CFG)

    if os.path.exists(CFG.best_warmstart_policy_by_acc_path):
        print("Loading best warm-start policy by accuracy...")
        policy.load_state_dict(torch.load(CFG.
    ↪best_warmstart_policy_by_acc_path, map_location=CFG.device))
    elif os.path.exists(CFG.best_warmstart_policy_path):
        print("Loading best warm-start policy by reward...")
        policy.load_state_dict(torch.load(CFG.best_warmstart_policy_path,
    ↪map_location=CFG.device))

    print("\nEvaluating deterministic policy right after warm-start...")

```

```

warm_det = evaluate_policy_deterministic(test_data, featurizer, policy, llm, CFG)
print(json.dumps({"warmstart_deterministic_eval": warm_det}, indent=2))

print("\nEvaluating sampled policy right after warm-start...")
warm_samp = evaluate_policy_sampled(test_data, featurizer, policy, llm, CFG)
print(json.dumps({"warmstart_sampled_eval": warm_samp}, indent=2))

train_policy_with_value(
    train_data=train_data, test_data=test_data,
    featurizer=featurizer, policy=policy, value_net=value_net,
    llm=llm, cfg=CFG, oracle_records=oracle_records,
)

print("Loading best saved policy/value for final evaluation...")
if os.path.exists(CFG.best_policy_by_acc_path):
    policy.load_state_dict(torch.load(CFG.best_policy_by_acc_path,
map_location=CFG.device))
elif os.path.exists(CFG.best_policy_path):
    policy.load_state_dict(torch.load(CFG.best_policy_path,
map_location=CFG.device))
if os.path.exists(CFG.best_value_path):
    value_net.load_state_dict(torch.load(CFG.best_value_path,
map_location=CFG.device))

print("\nFinal deterministic policy evaluation...")
final_det = evaluate_policy_deterministic(test_data, featurizer, policy, llm, CFG)
print(json.dumps({"final_deterministic_eval": final_det}, indent=2))

print("\nFinal sampled policy evaluation...")
final_samp = evaluate_policy_sampled(test_data, featurizer, policy, llm, CFG)
print(json.dumps({"final_sampled_eval": final_samp}, indent=2))

torch.save(policy.state_dict(), CFG.final_policy_path)
torch.save(value_net.state_dict(), CFG.final_value_path)
print(f"\nSaved final policy to {CFG.final_policy_path}")
print(f"Saved final value net to {CFG.final_value_path}")

# Final comparison table
print("\n" + "=" * 75)
print("FINAL RESULTS COMPARISON")
print("=" * 75)
print(f"{'Method':<35} {'Acc':>6} {'AvgTokens':>10} {'AvgReward':>10}")
print("-" * 75)

```

```

for b, res in baseline_results.items():
    print(f" Fixed budget {b:<22} {res['accuracy']:>6.4f} "
          f"{res['avg_thinking_tokens']:>10.0f} {res['avg_reward']:>10.4f}")
    print(f" Policy deterministic {final_det['accuracy']:>6.4f} "
          f"{final_det['avg_thinking_tokens']:>10.0f} {final_det['avg_reward']:>10.4f}")
    print(f" Policy sampled {final_samp['accuracy']:>6.4f} "
          f"{final_samp['avg_thinking_tokens']:>10.0f} {final_samp['avg_reward']:>10.4f}")
    print("=" * 75)

main()

# AI Disclosure: I have taken the help of Claude while writing this code.

```

/opt/conda/lib/python3.13/site-packages/tqdm/auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html

from .autonotebook import tqdm as notebook_tqdm
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

torch version: 2.11.0+cu128
cuda available: True
cuda device count: 1
gpu name: NVIDIA H200 NVL

Clearing old cache at: ./cache_rl_v10
Using device: cuda
Loading GSM8K...
Train subset: 300 | Test subset: 100
Loading featurizer...

Loading weights: 100%| | 103/103 [00:00<00:00, 778.50it/s]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key | Status | |
-----+-----+--
embeddings.position_ids | UNEXPECTED | |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.

Loading policy and value network...
Loading LLM...

`torch\_dtype` is deprecated! Use `dtype` instead!
Loading weights: 100%| | 338/338 [00:12<00:00, 27.93it/s]


```
=====
=====
DEBUG EXAMPLES WITH FIXED THINKING BUDGET = 176
=====
=====
```

[Example 1]

QUESTION: Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did Natalia sell altogether in April and May?

THINKING TEXT:

To find out how many clips Natalia sold in total over April and May, we need to follow these steps:

1. Determine how many clips Natalia sold in May. Since she sold half as many clips in May as she did in April, we calculate:

```
\[
\text{Clips sold in May} = \frac{\text{Clips sold in April}}{2}
\]
```

Given that she sold 48 clips in April, we substitute this value into the equation:

```
\[
\text{Clips sold in May} = \frac{48}{2} = 24
\]
```

2. Add the number of clips sold in April to the number sold in May to get the total for both months:

```
\[
\text{Total clips sold} = \text{Clips sold in April} + \text{Clips sold in May}
\]
```

ANSWER TEXT:

72

PARSED PREDICTION: 72

GOLD ANSWER : 72

CORRECT : 1

THINK TOKENS : 176

REWARD : 0.722

[Example 2]

QUESTION: Weng earns \$12 an hour for babysitting. Yesterday, she just did 50 minutes of babysitting. How much did she earn?

THINKING TEXT:

To find out how much Weng earned from babysitting, we need to calculate her earnings based on the rate at which she is paid (\$12 per hour) and the time she spent working (50 minutes). First, convert the time from minutes to hours since the pay rate is given in dollars per hour. There are 60 minutes in an hour, so 50 minutes is equal to $50/60 = 5/6$ hours. Then multiply this hourly rate by the number of hours worked to get the total earnings. So, $\$12/\text{hour} * 5/6 \text{ hours} = \10 . Therefore, Weng earned \$10 from yesterday's babysitting.

ANSWER TEXT:

10

PARSED PREDICTION: 10

GOLD ANSWER : 10

CORRECT : 1

THINK TOKENS : 176

REWARD : 0.722

[Example 3]

QUESTION: Betty is saving money for a new wallet which costs \$100. Betty has only half of the money she needs. Her parents decided to give her \$15 for that purpose, and her grandparents twice as much as her parents. How much more money does Betty need to buy the wallet?

THINKING TEXT:

First, we determine how much money Betty currently has. Since she has only half of what she needs, we calculate:

$$\text{Betty's current savings} = \frac{\$100}{2} = \$50$$

Next, we account for the additional money given by her parents. They gave her \$15, so now we add this amount to Betty's current savings:

$$\text{Total after parents' gift} = \$50 + \$15 = \$65$$

Then, we consider the contribution from her grandparents. The problem states they gave her twice as much as her parents, so:

$$\text{Grandparents' contribution} = 2 \times \$15 = \$30$$

Adding this to Betty's total gives us:

$$\text{Total with grandparents' contribution} =$$

ANSWER TEXT:

```

PARSED PREDICTION: 45
GOLD ANSWER       : 5
CORRECT           : 0
THINK TOKENS      : 176
REWARD            : -0.528

```

```

=====
FIXED BUDGET BASELINES ON TEST SET
=====

```

Evaluating fixed budget = 96 on 100 examples...

```

Fixed 96 | 10/100 | Acc=0.4000 | AvgTokens=96.0 | AvgReward=0.2120
Fixed 96 | 20/100 | Acc=0.2500 | AvgTokens=96.0 | AvgReward=0.0245
Fixed 96 | 30/100 | Acc=0.2333 | AvgTokens=96.0 | AvgReward=0.0037
Fixed 96 | 40/100 | Acc=0.2250 | AvgTokens=96.0 | AvgReward=-0.0068
Fixed 96 | 50/100 | Acc=0.2400 | AvgTokens=96.0 | AvgReward=0.0120
Fixed 96 | 60/100 | Acc=0.2667 | AvgTokens=96.0 | AvgReward=0.0453
Fixed 96 | 70/100 | Acc=0.2571 | AvgTokens=96.0 | AvgReward=0.0334
Fixed 96 | 80/100 | Acc=0.2500 | AvgTokens=96.0 | AvgReward=0.0245
Fixed 96 | 90/100 | Acc=0.2556 | AvgTokens=96.0 | AvgReward=0.0314
Fixed 96 | 100/100 | Acc=0.2600 | AvgTokens=96.0 | AvgReward=0.0370
Fixed 96 | Acc=0.2600 | AvgTokens=96 | AvgReward=0.0370

```

Evaluating fixed budget = 128 on 100 examples...

```

Fixed 128 | 10/100 | Acc=0.4000 | AvgTokens=128.0 | AvgReward=0.1160
Fixed 128 | 20/100 | Acc=0.3000 | AvgTokens=128.0 | AvgReward=-0.0090
Fixed 128 | 30/100 | Acc=0.3000 | AvgTokens=128.0 | AvgReward=-0.0090
Fixed 128 | 40/100 | Acc=0.2500 | AvgTokens=128.0 | AvgReward=-0.0715
Fixed 128 | 50/100 | Acc=0.2800 | AvgTokens=128.0 | AvgReward=-0.0340
Fixed 128 | 60/100 | Acc=0.3000 | AvgTokens=128.0 | AvgReward=-0.0090
Fixed 128 | 70/100 | Acc=0.3143 | AvgTokens=128.0 | AvgReward=0.0089
Fixed 128 | 80/100 | Acc=0.3000 | AvgTokens=128.0 | AvgReward=-0.0090
Fixed 128 | 90/100 | Acc=0.3222 | AvgTokens=128.0 | AvgReward=0.0188
Fixed 128 | 100/100 | Acc=0.3300 | AvgTokens=128.0 | AvgReward=0.0285
Fixed 128 | Acc=0.3300 | AvgTokens=128 | AvgReward=0.0285

```

Evaluating fixed budget = 144 on 100 examples...

```

Fixed 144 | 10/100 | Acc=0.5000 | AvgTokens=144.0 | AvgReward=0.1930
Fixed 144 | 20/100 | Acc=0.3000 | AvgTokens=144.0 | AvgReward=-0.0570
Fixed 144 | 30/100 | Acc=0.3000 | AvgTokens=144.0 | AvgReward=-0.0570
Fixed 144 | 40/100 | Acc=0.2500 | AvgTokens=144.0 | AvgReward=-0.1195
Fixed 144 | 50/100 | Acc=0.2800 | AvgTokens=144.0 | AvgReward=-0.0820
Fixed 144 | 60/100 | Acc=0.3167 | AvgTokens=144.0 | AvgReward=-0.0362
Fixed 144 | 70/100 | Acc=0.3571 | AvgTokens=144.0 | AvgReward=0.0144

```

Fixed 144	80/100	Acc=0.3500	AvgTokens=144.0	AvgReward=0.0055
Fixed 144	90/100	Acc=0.3444	AvgTokens=144.0	AvgReward=-0.0014
Fixed 144	100/100	Acc=0.3700	AvgTokens=144.0	AvgReward=0.0305
Fixed 144		Acc=0.3700	AvgTokens=144	AvgReward=0.0305

Evaluating fixed budget = 160 on 100 examples...

Fixed 160	10/100	Acc=0.4000	AvgTokens=160.0	AvgReward=0.0200
Fixed 160	20/100	Acc=0.4000	AvgTokens=160.0	AvgReward=0.0200
Fixed 160	30/100	Acc=0.4667	AvgTokens=160.0	AvgReward=0.1033
Fixed 160	40/100	Acc=0.4500	AvgTokens=160.0	AvgReward=0.0825
Fixed 160	50/100	Acc=0.4600	AvgTokens=160.0	AvgReward=0.0950
Fixed 160	60/100	Acc=0.4833	AvgTokens=160.0	AvgReward=0.1242
Fixed 160	70/100	Acc=0.5143	AvgTokens=160.0	AvgReward=0.1629
Fixed 160	80/100	Acc=0.4875	AvgTokens=160.0	AvgReward=0.1294
Fixed 160	90/100	Acc=0.4667	AvgTokens=160.0	AvgReward=0.1033
Fixed 160	100/100	Acc=0.4900	AvgTokens=160.0	AvgReward=0.1325
Fixed 160		Acc=0.4900	AvgTokens=160	AvgReward=0.1325

Evaluating fixed budget = 176 on 100 examples...

Fixed 176	10/100	Acc=0.5000	AvgTokens=176.0	AvgReward=0.0970
Fixed 176	20/100	Acc=0.4500	AvgTokens=176.0	AvgReward=0.0345
Fixed 176	30/100	Acc=0.5000	AvgTokens=176.0	AvgReward=0.0970
Fixed 176	40/100	Acc=0.5250	AvgTokens=176.0	AvgReward=0.1282
Fixed 176	50/100	Acc=0.5000	AvgTokens=176.0	AvgReward=0.0970
Fixed 176	60/100	Acc=0.5167	AvgTokens=176.0	AvgReward=0.1178
Fixed 176	70/100	Acc=0.5286	AvgTokens=176.0	AvgReward=0.1327
Fixed 176	80/100	Acc=0.5125	AvgTokens=176.0	AvgReward=0.1126
Fixed 176	90/100	Acc=0.5000	AvgTokens=176.0	AvgReward=0.0970
Fixed 176	100/100	Acc=0.5300	AvgTokens=176.0	AvgReward=0.1345
Fixed 176		Acc=0.5300	AvgTokens=176	AvgReward=0.1345

Evaluating fixed budget = 192 on 100 examples...

Fixed 192	10/100	Acc=0.4000	AvgTokens=192.0	AvgReward=-0.0760
Fixed 192	20/100	Acc=0.3500	AvgTokens=192.0	AvgReward=-0.1385
Fixed 192	30/100	Acc=0.4333	AvgTokens=192.0	AvgReward=-0.0343
Fixed 192	40/100	Acc=0.5000	AvgTokens=192.0	AvgReward=0.0490
Fixed 192	50/100	Acc=0.5000	AvgTokens=192.0	AvgReward=0.0490
Fixed 192	60/100	Acc=0.5000	AvgTokens=192.0	AvgReward=0.0490
Fixed 192	70/100	Acc=0.5143	AvgTokens=192.0	AvgReward=0.0669
Fixed 192	80/100	Acc=0.5125	AvgTokens=192.0	AvgReward=0.0646
Fixed 192	90/100	Acc=0.5000	AvgTokens=192.0	AvgReward=0.0490
Fixed 192	100/100	Acc=0.5200	AvgTokens=192.0	AvgReward=0.0740
Fixed 192		Acc=0.5200	AvgTokens=192	AvgReward=0.0740

Evaluating fixed budget = 220 on 100 examples...

Fixed 220	10/100	Acc=0.5000	AvgTokens=220.0	AvgReward=-0.0350
Fixed 220	20/100	Acc=0.4000	AvgTokens=220.0	AvgReward=-0.1600
Fixed 220	30/100	Acc=0.4667	AvgTokens=220.0	AvgReward=-0.0767

Fixed 220		40/100		Acc=0.4750		AvgTokens=220.0		AvgReward=-0.0663
Fixed 220		50/100		Acc=0.4800		AvgTokens=220.0		AvgReward=-0.0600
Fixed 220		60/100		Acc=0.4833		AvgTokens=220.0		AvgReward=-0.0558
Fixed 220		70/100		Acc=0.5000		AvgTokens=220.0		AvgReward=-0.0350
Fixed 220		80/100		Acc=0.5125		AvgTokens=220.0		AvgReward=-0.0194
Fixed 220		90/100		Acc=0.5000		AvgTokens=220.0		AvgReward=-0.0350
Fixed 220		100/100		Acc=0.5200		AvgTokens=220.0		AvgReward=-0.0100
Fixed 220		Acc=0.5200		AvgTokens=220		AvgReward=-0.0100		

Building oracle budget labels...

Oracle		10/300		AvgBestBudget=116.8		AvgBestReward=0.3996		FracSolvable=0.600
Oracle		20/300		AvgBestBudget=116.0		AvgBestReward=0.2770		FracSolvable=0.500
Oracle		30/300		AvgBestBudget=116.3		AvgBestReward=0.3179		FracSolvable=0.533
Oracle		40/300		AvgBestBudget=120.0		AvgBestReward=0.3900		FracSolvable=0.600
Oracle		50/300		AvgBestBudget=118.7		AvgBestReward=0.3938		FracSolvable=0.600
Oracle		60/300		AvgBestBudget=123.5		AvgBestReward=0.4213		FracSolvable=0.633
Oracle		70/300		AvgBestBudget=122.9		AvgBestReward=0.3993		FracSolvable=0.614
Oracle		80/300		AvgBestBudget=124.3		AvgBestReward=0.4083		FracSolvable=0.625
Oracle		90/300		AvgBestBudget=123.0		AvgBestReward=0.4225		FracSolvable=0.633
Oracle		100/300		AvgBestBudget=122.5		AvgBestReward=0.4200		FracSolvable=0.630
Oracle		110/300		AvgBestBudget=123.0		AvgBestReward=0.4377		FracSolvable=0.645
Oracle		120/300		AvgBestBudget=123.8		AvgBestReward=0.4514		FracSolvable=0.658
Oracle		130/300		AvgBestBudget=123.1		AvgBestReward=0.4385		FracSolvable=0.646
Oracle		140/300		AvgBestBudget=122.5		AvgBestReward=0.4450		FracSolvable=0.650
Oracle		150/300		AvgBestBudget=122.5		AvgBestReward=0.4493		FracSolvable=0.653
Oracle		160/300		AvgBestBudget=123.6		AvgBestReward=0.4651		FracSolvable=0.669
Oracle		170/300		AvgBestBudget=123.3		AvgBestReward=0.4537		FracSolvable=0.659
Oracle		180/300		AvgBestBudget=122.5		AvgBestReward=0.4520		FracSolvable=0.656
Oracle		190/300		AvgBestBudget=121.9		AvgBestReward=0.4435		FracSolvable=0.647

Oracle | 200/300 | AvgBestBudget=121.2 | AvgBestReward=0.4364 |
 FracSolvable=0.640
 Oracle | 210/300 | AvgBestBudget=121.1 | AvgBestReward=0.4403 |
 FracSolvable=0.643
 Oracle | 220/300 | AvgBestBudget=120.7 | AvgBestReward=0.4278 |
 FracSolvable=0.632
 Oracle | 230/300 | AvgBestBudget=120.6 | AvgBestReward=0.4262 |
 FracSolvable=0.630
 Oracle | 240/300 | AvgBestBudget=119.9 | AvgBestReward=0.4214 |
 FracSolvable=0.625
 Oracle | 250/300 | AvgBestBudget=120.5 | AvgBestReward=0.4235 |
 FracSolvable=0.628
 Oracle | 260/300 | AvgBestBudget=120.6 | AvgBestReward=0.4362 |
 FracSolvable=0.638
 Oracle | 270/300 | AvgBestBudget=120.1 | AvgBestReward=0.4407 |
 FracSolvable=0.641
 Oracle | 280/300 | AvgBestBudget=120.3 | AvgBestReward=0.4338 |
 FracSolvable=0.636
 Oracle | 290/300 | AvgBestBudget=120.0 | AvgBestReward=0.4288 |
 FracSolvable=0.631
 Oracle | 300/300 | AvgBestBudget=120.0 | AvgBestReward=0.4234 |
 FracSolvable=0.627
 Saved oracle labels to oracle_budget_labels_v10.json

Oracle stats: mean=120.0, std=34.7, solvable=0.627

budget=96: 171 questions (57.0%)
 budget=112: 31 questions (10.3%)
 budget=128: 20 questions (6.7%)
 budget=144: 15 questions (5.0%)
 budget=160: 14 questions (4.7%)
 budget=168: 10 questions (3.3%)
 budget=176: 10 questions (3.3%)
 budget=184: 6 questions (2.0%)
 budget=192: 11 questions (3.7%)
 budget=200: 6 questions (2.0%)
 budget=210: 3 questions (1.0%)
 budget=220: 3 questions (1.0%)

Starting supervised warm-start from oracle budgets...

[Warmstart 1/15] Step 20/300 | Loss=0.154908 | PredAvg=103.1 | TargetAvg=114.9 |
 MAE=36.20
 [Warmstart 1/15] Step 40/300 | Loss=0.172706 | PredAvg=111.8 | TargetAvg=119.5 |
 MAE=40.02
 [Warmstart 1/15] Step 60/300 | Loss=0.179605 | PredAvg=114.6 | TargetAvg=123.8 |
 MAE=41.60
 [Warmstart 1/15] Step 80/300 | Loss=0.161137 | PredAvg=111.2 | TargetAvg=122.0 |
 MAE=40.05
 [Warmstart 1/15] Step 100/300 | Loss=0.150538 | PredAvg=113.6 | TargetAvg=122.0

| MAE=39.99
 [Warmstart 1/15] Step 120/300 | Loss=0.148977 | PredAvg=111.6 | TargetAvg=121.6
 | MAE=38.99
 [Warmstart 1/15] Step 140/300 | Loss=0.146262 | PredAvg=112.7 | TargetAvg=121.0
 | MAE=38.54
 [Warmstart 1/15] Step 160/300 | Loss=0.135306 | PredAvg=111.6 | TargetAvg=119.7
 | MAE=37.17
 [Warmstart 1/15] Step 180/300 | Loss=0.137618 | PredAvg=114.8 | TargetAvg=121.3
 | MAE=38.24
 [Warmstart 1/15] Step 200/300 | Loss=0.131221 | PredAvg=113.1 | TargetAvg=119.9
 | MAE=36.70
 [Warmstart 1/15] Step 220/300 | Loss=0.130432 | PredAvg=114.0 | TargetAvg=120.6
 | MAE=36.99
 [Warmstart 1/15] Step 240/300 | Loss=0.129698 | PredAvg=114.9 | TargetAvg=120.9
 | MAE=37.14
 [Warmstart 1/15] Step 260/300 | Loss=0.126257 | PredAvg=114.4 | TargetAvg=120.1
 | MAE=36.21
 [Warmstart 1/15] Step 280/300 | Loss=0.121348 | PredAvg=113.2 | TargetAvg=119.6
 | MAE=35.38
 [Warmstart 1/15] Step 300/300 | Loss=0.120802 | PredAvg=115.4 | TargetAvg=120.0
 | MAE=36.14
 Warmstart epoch 1 summary | Loss=0.120802 | PredAvg=115.4 | TargetAvg=120.0

Warm-start deterministic evaluation on test set...

Det eval | 10/100 | Acc=0.4000 | AvgBudget=128.40 | StdBudget=17.73
 Det eval | 20/100 | Acc=0.3000 | AvgBudget=128.70 | StdBudget=14.80
 Det eval | 30/100 | Acc=0.2333 | AvgBudget=130.23 | StdBudget=13.07
 Det eval | 40/100 | Acc=0.2250 | AvgBudget=132.40 | StdBudget=12.97
 Det eval | 50/100 | Acc=0.2600 | AvgBudget=130.56 | StdBudget=14.77
 Det eval | 60/100 | Acc=0.3167 | AvgBudget=130.30 | StdBudget=15.00
 Det eval | 70/100 | Acc=0.3143 | AvgBudget=130.36 | StdBudget=14.62
 Det eval | 80/100 | Acc=0.3000 | AvgBudget=130.34 | StdBudget=14.48
 Det eval | 90/100 | Acc=0.2889 | AvgBudget=130.79 | StdBudget=14.96
 Det eval | 100/100 | Acc=0.2900 | AvgBudget=130.50 | StdBudget=15.00
 Saved best warm-start policy by reward = -0.0290

Saved best warm-start policy by accuracy = 0.2900

[Warmstart 2/15] Step 20/300 | Loss=0.075487 | PredAvg=111.1 | TargetAvg=110.2 |
 MAE=26.90
 [Warmstart 2/15] Step 40/300 | Loss=0.059978 | PredAvg=109.0 | TargetAvg=111.3 |
 MAE=25.73
 [Warmstart 2/15] Step 60/300 | Loss=0.087388 | PredAvg=108.3 | TargetAvg=111.1 |
 MAE=28.37
 [Warmstart 2/15] Step 80/300 | Loss=0.086358 | PredAvg=109.7 | TargetAvg=113.0 |
 MAE=28.96
 [Warmstart 2/15] Step 100/300 | Loss=0.100688 | PredAvg=109.3 | TargetAvg=116.5
 | MAE=29.70

[Warmstart 2/15] Step 120/300 | Loss=0.096563 | PredAvg=114.9 | TargetAvg=117.2
 | MAE=31.76
 [Warmstart 2/15] Step 140/300 | Loss=0.099584 | PredAvg=117.4 | TargetAvg=118.6
 | MAE=33.07
 [Warmstart 2/15] Step 160/300 | Loss=0.110185 | PredAvg=116.7 | TargetAvg=118.9
 | MAE=34.03
 [Warmstart 2/15] Step 180/300 | Loss=0.112245 | PredAvg=116.2 | TargetAvg=119.6
 | MAE=34.28
 [Warmstart 2/15] Step 200/300 | Loss=0.111077 | PredAvg=117.3 | TargetAvg=119.2
 | MAE=34.48
 [Warmstart 2/15] Step 220/300 | Loss=0.113409 | PredAvg=115.8 | TargetAvg=118.7
 | MAE=34.16
 [Warmstart 2/15] Step 240/300 | Loss=0.122698 | PredAvg=114.5 | TargetAvg=118.8
 | MAE=34.33
 [Warmstart 2/15] Step 260/300 | Loss=0.123378 | PredAvg=115.4 | TargetAvg=119.8
 | MAE=34.38
 [Warmstart 2/15] Step 280/300 | Loss=0.116582 | PredAvg=115.9 | TargetAvg=119.4
 | MAE=33.39
 [Warmstart 2/15] Step 300/300 | Loss=0.116554 | PredAvg=116.6 | TargetAvg=120.0
 | MAE=33.30
 Warmstart epoch 2 summary | Loss=0.116554 | PredAvg=116.6 | TargetAvg=120.0

Warm-start deterministic evaluation on test set...

Det eval | 10/100 | Acc=0.4000 | AvgBudget=124.50 | StdBudget=18.09
 Det eval | 20/100 | Acc=0.3000 | AvgBudget=124.70 | StdBudget=15.17
 Det eval | 30/100 | Acc=0.3000 | AvgBudget=126.33 | StdBudget=13.42
 Det eval | 40/100 | Acc=0.2750 | AvgBudget=128.68 | StdBudget=13.33
 Det eval | 50/100 | Acc=0.3000 | AvgBudget=126.78 | StdBudget=15.12
 Det eval | 60/100 | Acc=0.3333 | AvgBudget=126.50 | StdBudget=15.43
 Det eval | 70/100 | Acc=0.3286 | AvgBudget=126.57 | StdBudget=15.03
 Det eval | 80/100 | Acc=0.3125 | AvgBudget=126.53 | StdBudget=14.86
 Det eval | 90/100 | Acc=0.3000 | AvgBudget=127.03 | StdBudget=15.37
 Det eval | 100/100 | Acc=0.3100 | AvgBudget=126.75 | StdBudget=15.43

Saved best warm-start policy by reward = 0.0072

Saved best warm-start policy by accuracy = 0.3100

[Warmstart 3/15] Step 20/300 | Loss=0.067994 | PredAvg=128.4 | TargetAvg=111.3 |
 MAE=30.85
 [Warmstart 3/15] Step 40/300 | Loss=0.093400 | PredAvg=120.7 | TargetAvg=120.5 |
 MAE=30.48
 [Warmstart 3/15] Step 60/300 | Loss=0.090293 | PredAvg=122.0 | TargetAvg=119.3 |
 MAE=29.00
 [Warmstart 3/15] Step 80/300 | Loss=0.091458 | PredAvg=118.8 | TargetAvg=118.4 |
 MAE=28.44
 [Warmstart 3/15] Step 100/300 | Loss=0.092253 | PredAvg=116.0 | TargetAvg=117.6
 | MAE=28.27
 [Warmstart 3/15] Step 120/300 | Loss=0.091151 | PredAvg=113.9 | TargetAvg=116.5

| MAE=27.52
 [Warmstart 3/15] Step 140/300 | Loss=0.094664 | PredAvg=114.6 | TargetAvg=119.2
 | MAE=28.46
 [Warmstart 3/15] Step 160/300 | Loss=0.096460 | PredAvg=119.0 | TargetAvg=120.7
 | MAE=30.24
 [Warmstart 3/15] Step 180/300 | Loss=0.092995 | PredAvg=118.8 | TargetAvg=119.1
 | MAE=30.74
 [Warmstart 3/15] Step 200/300 | Loss=0.094537 | PredAvg=118.2 | TargetAvg=119.0
 | MAE=30.73
 [Warmstart 3/15] Step 220/300 | Loss=0.092728 | PredAvg=119.1 | TargetAvg=119.2
 | MAE=30.82
 [Warmstart 3/15] Step 240/300 | Loss=0.097328 | PredAvg=118.9 | TargetAvg=119.8
 | MAE=31.09
 [Warmstart 3/15] Step 260/300 | Loss=0.098593 | PredAvg=118.2 | TargetAvg=119.8
 | MAE=30.80
 [Warmstart 3/15] Step 280/300 | Loss=0.099475 | PredAvg=118.4 | TargetAvg=119.5
 | MAE=31.03
 [Warmstart 3/15] Step 300/300 | Loss=0.105908 | PredAvg=117.6 | TargetAvg=120.0
 | MAE=31.30
 Warmstart epoch 3 summary | Loss=0.105908 | PredAvg=117.6 | TargetAvg=120.0

Warm-start deterministic evaluation on test set...

Det eval | 10/100 | Acc=0.4000 | AvgBudget=123.60 | StdBudget=16.89
 Det eval | 20/100 | Acc=0.3000 | AvgBudget=123.80 | StdBudget=14.20
 Det eval | 30/100 | Acc=0.2667 | AvgBudget=125.17 | StdBudget=12.47
 Det eval | 40/100 | Acc=0.2750 | AvgBudget=127.17 | StdBudget=12.33
 Det eval | 50/100 | Acc=0.3000 | AvgBudget=125.46 | StdBudget=14.08
 Det eval | 60/100 | Acc=0.3167 | AvgBudget=125.22 | StdBudget=14.38
 Det eval | 70/100 | Acc=0.3143 | AvgBudget=125.31 | StdBudget=13.96
 Det eval | 80/100 | Acc=0.3000 | AvgBudget=125.30 | StdBudget=13.83
 Det eval | 90/100 | Acc=0.2778 | AvgBudget=125.77 | StdBudget=14.29
 Det eval | 100/100 | Acc=0.2800 | AvgBudget=125.49 | StdBudget=14.32
 [Warmstart 4/15] Step 20/300 | Loss=0.146637 | PredAvg=119.7 | TargetAvg=115.6 |
 MAE=29.50
 [Warmstart 4/15] Step 40/300 | Loss=0.104032 | PredAvg=116.0 | TargetAvg=116.8 |
 MAE=27.25
 [Warmstart 4/15] Step 60/300 | Loss=0.105101 | PredAvg=117.2 | TargetAvg=116.3 |
 MAE=28.35
 [Warmstart 4/15] Step 80/300 | Loss=0.107321 | PredAvg=118.2 | TargetAvg=116.4 |
 MAE=30.20
 [Warmstart 4/15] Step 100/300 | Loss=0.101037 | PredAvg=116.2 | TargetAvg=117.3
 | MAE=28.87
 [Warmstart 4/15] Step 120/300 | Loss=0.096178 | PredAvg=118.0 | TargetAvg=117.9
 | MAE=29.64
 [Warmstart 4/15] Step 140/300 | Loss=0.099565 | PredAvg=120.4 | TargetAvg=120.7
 | MAE=30.69
 [Warmstart 4/15] Step 160/300 | Loss=0.098562 | PredAvg=122.4 | TargetAvg=120.5
 | MAE=32.07

[Warmstart 4/15] Step 180/300 | Loss=0.094253 | PredAvg=122.6 | TargetAvg=120.4
 | MAE=31.89
 [Warmstart 4/15] Step 200/300 | Loss=0.090979 | PredAvg=122.0 | TargetAvg=120.7
 | MAE=31.03
 [Warmstart 4/15] Step 220/300 | Loss=0.093020 | PredAvg=121.6 | TargetAvg=120.3
 | MAE=31.08
 [Warmstart 4/15] Step 240/300 | Loss=0.092984 | PredAvg=121.0 | TargetAvg=120.1
 | MAE=30.70
 [Warmstart 4/15] Step 260/300 | Loss=0.099421 | PredAvg=119.9 | TargetAvg=121.4
 | MAE=31.12
 [Warmstart 4/15] Step 280/300 | Loss=0.098576 | PredAvg=120.7 | TargetAvg=121.2
 | MAE=31.15
 [Warmstart 4/15] Step 300/300 | Loss=0.093318 | PredAvg=119.9 | TargetAvg=120.0
 | MAE=30.11
 Warmstart epoch 4 summary | Loss=0.093318 | PredAvg=119.9 | TargetAvg=120.0

Warm-start deterministic evaluation on test set...

Det eval | 10/100 | Acc=0.4000 | AvgBudget=98.30 | StdBudget=17.94
 Det eval | 20/100 | Acc=0.3000 | AvgBudget=97.50 | StdBudget=15.39
 Det eval | 30/100 | Acc=0.3000 | AvgBudget=98.67 | StdBudget=13.69
 Det eval | 40/100 | Acc=0.3000 | AvgBudget=101.08 | StdBudget=13.94
 Det eval | 50/100 | Acc=0.2800 | AvgBudget=99.60 | StdBudget=14.81
 Det eval | 60/100 | Acc=0.2833 | AvgBudget=99.43 | StdBudget=15.18
 Det eval | 70/100 | Acc=0.2571 | AvgBudget=99.43 | StdBudget=14.88
 Det eval | 80/100 | Acc=0.2375 | AvgBudget=99.39 | StdBudget=14.69
 Det eval | 90/100 | Acc=0.2222 | AvgBudget=100.06 | StdBudget=15.54
 Det eval | 100/100 | Acc=0.2200 | AvgBudget=99.76 | StdBudget=15.63
 [Warmstart 5/15] Step 20/300 | Loss=0.087157 | PredAvg=107.6 | TargetAvg=115.2 |
 MAE=22.60
 [Warmstart 5/15] Step 40/300 | Loss=0.070426 | PredAvg=101.0 | TargetAvg=113.6 |
 MAE=22.85
 [Warmstart 5/15] Step 60/300 | Loss=0.101877 | PredAvg=106.7 | TargetAvg=118.6 |
 MAE=28.12
 [Warmstart 5/15] Step 80/300 | Loss=0.109228 | PredAvg=108.5 | TargetAvg=119.7 |
 MAE=28.62
 [Warmstart 5/15] Step 100/300 | Loss=0.101661 | PredAvg=110.5 | TargetAvg=118.3
 | MAE=28.88
 [Warmstart 5/15] Step 120/300 | Loss=0.108814 | PredAvg=110.8 | TargetAvg=120.2
 | MAE=29.07
 [Warmstart 5/15] Step 140/300 | Loss=0.103976 | PredAvg=112.9 | TargetAvg=119.7
 | MAE=29.29
 [Warmstart 5/15] Step 160/300 | Loss=0.100957 | PredAvg=113.4 | TargetAvg=119.3
 | MAE=29.61
 [Warmstart 5/15] Step 180/300 | Loss=0.095962 | PredAvg=114.2 | TargetAvg=119.8
 | MAE=29.08
 [Warmstart 5/15] Step 200/300 | Loss=0.089689 | PredAvg=115.4 | TargetAvg=118.9
 | MAE=29.20
 [Warmstart 5/15] Step 220/300 | Loss=0.092219 | PredAvg=116.5 | TargetAvg=119.2

| MAE=29.70
 [Warmstart 5/15] Step 240/300 | Loss=0.091040 | PredAvg=115.3 | TargetAvg=118.5
 | MAE=29.15
 [Warmstart 5/15] Step 260/300 | Loss=0.092411 | PredAvg=115.3 | TargetAvg=118.9
 | MAE=29.36
 [Warmstart 5/15] Step 280/300 | Loss=0.098000 | PredAvg=114.0 | TargetAvg=118.9
 | MAE=29.62
 [Warmstart 5/15] Step 300/300 | Loss=0.105319 | PredAvg=113.7 | TargetAvg=120.0
 | MAE=30.09
 Warmstart epoch 5 summary | Loss=0.105319 | PredAvg=113.7 | TargetAvg=120.0

Warm-start deterministic evaluation on test set...

Det eval | 10/100 | Acc=0.4000 | AvgBudget=126.60 | StdBudget=12.51
 Det eval | 20/100 | Acc=0.3000 | AvgBudget=126.80 | StdBudget=10.33
 Det eval | 30/100 | Acc=0.2667 | AvgBudget=127.70 | StdBudget=8.97
 Det eval | 40/100 | Acc=0.2250 | AvgBudget=128.72 | StdBudget=8.64
 Det eval | 50/100 | Acc=0.2600 | AvgBudget=127.74 | StdBudget=9.98
 Det eval | 60/100 | Acc=0.2833 | AvgBudget=127.55 | StdBudget=10.16
 Det eval | 70/100 | Acc=0.3000 | AvgBudget=127.76 | StdBudget=9.83
 Det eval | 80/100 | Acc=0.2875 | AvgBudget=127.78 | StdBudget=9.84
 Det eval | 90/100 | Acc=0.2889 | AvgBudget=128.06 | StdBudget=10.15
 Det eval | 100/100 | Acc=0.2900 | AvgBudget=127.87 | StdBudget=10.16
 [Warmstart 6/15] Step 20/300 | Loss=0.062140 | PredAvg=125.2 | TargetAvg=110.5 |
 MAE=27.15
 [Warmstart 6/15] Step 40/300 | Loss=0.080241 | PredAvg=114.4 | TargetAvg=114.2 |
 MAE=27.23
 [Warmstart 6/15] Step 60/300 | Loss=0.112874 | PredAvg=111.8 | TargetAvg=118.6 |
 MAE=30.22
 [Warmstart 6/15] Step 80/300 | Loss=0.098845 | PredAvg=114.9 | TargetAvg=117.3 |
 MAE=29.43
 [Warmstart 6/15] Step 100/300 | Loss=0.114253 | PredAvg=112.0 | TargetAvg=118.0
 | MAE=30.16
 [Warmstart 6/15] Step 120/300 | Loss=0.125999 | PredAvg=111.9 | TargetAvg=120.5
 | MAE=30.87
 [Warmstart 6/15] Step 140/300 | Loss=0.118244 | PredAvg=115.2 | TargetAvg=120.1
 | MAE=31.83
 [Warmstart 6/15] Step 160/300 | Loss=0.110404 | PredAvg=116.2 | TargetAvg=120.0
 | MAE=31.22
 [Warmstart 6/15] Step 180/300 | Loss=0.107558 | PredAvg=115.6 | TargetAvg=119.6
 | MAE=30.53
 [Warmstart 6/15] Step 200/300 | Loss=0.105613 | PredAvg=114.9 | TargetAvg=119.4
 | MAE=29.90
 [Warmstart 6/15] Step 220/300 | Loss=0.107011 | PredAvg=114.9 | TargetAvg=120.3
 | MAE=29.76
 [Warmstart 6/15] Step 240/300 | Loss=0.104035 | PredAvg=115.2 | TargetAvg=119.4
 | MAE=29.82
 [Warmstart 6/15] Step 260/300 | Loss=0.100537 | PredAvg=114.6 | TargetAvg=118.5
 | MAE=28.96

[Warmstart 6/15] Step 280/300 | Loss=0.100379 | PredAvg=114.0 | TargetAvg=118.7
| MAE=28.77

[Warmstart 6/15] Step 300/300 | Loss=0.102431 | PredAvg=114.7 | TargetAvg=120.0
| MAE=29.08

Warmstart epoch 6 summary | Loss=0.102431 | PredAvg=114.7 | TargetAvg=120.0

Warm-start deterministic evaluation on test set...

Det eval | 10/100 | Acc=0.4000 | AvgBudget=136.60 | StdBudget=8.15

Det eval | 20/100 | Acc=0.3000 | AvgBudget=136.70 | StdBudget=6.58

Det eval | 30/100 | Acc=0.3000 | AvgBudget=137.13 | StdBudget=5.68

Det eval | 40/100 | Acc=0.2750 | AvgBudget=137.40 | StdBudget=5.40

Det eval | 50/100 | Acc=0.2800 | AvgBudget=137.02 | StdBudget=6.13

Det eval | 60/100 | Acc=0.3333 | AvgBudget=136.95 | StdBudget=6.20

Det eval | 70/100 | Acc=0.3429 | AvgBudget=137.17 | StdBudget=6.00

Det eval | 80/100 | Acc=0.3375 | AvgBudget=137.20 | StdBudget=6.12

Det eval | 90/100 | Acc=0.3333 | AvgBudget=137.29 | StdBudget=6.27

Det eval | 100/100 | Acc=0.3600 | AvgBudget=137.19 | StdBudget=6.27

Saved best warm-start policy by reward = 0.0384

Saved best warm-start policy by accuracy = 0.3600

[Warmstart 7/15] Step 20/300 | Loss=0.085513 | PredAvg=141.5 | TargetAvg=130.6 |
MAE=34.90

[Warmstart 7/15] Step 40/300 | Loss=0.089239 | PredAvg=131.7 | TargetAvg=122.3 |
MAE=34.02

[Warmstart 7/15] Step 60/300 | Loss=0.089807 | PredAvg=125.1 | TargetAvg=122.3 |
MAE=32.03

[Warmstart 7/15] Step 80/300 | Loss=0.093538 | PredAvg=125.4 | TargetAvg=122.1 |
MAE=32.52

[Warmstart 7/15] Step 100/300 | Loss=0.092625 | PredAvg=121.4 | TargetAvg=121.1
| MAE=30.87

[Warmstart 7/15] Step 120/300 | Loss=0.092560 | PredAvg=119.6 | TargetAvg=120.9
| MAE=30.79

[Warmstart 7/15] Step 140/300 | Loss=0.085816 | PredAvg=118.3 | TargetAvg=119.1
| MAE=29.06

[Warmstart 7/15] Step 160/300 | Loss=0.092824 | PredAvg=115.7 | TargetAvg=118.8
| MAE=28.84

[Warmstart 7/15] Step 180/300 | Loss=0.087837 | PredAvg=115.8 | TargetAvg=118.5
| MAE=27.93

[Warmstart 7/15] Step 200/300 | Loss=0.090736 | PredAvg=116.7 | TargetAvg=120.4
| MAE=28.61

[Warmstart 7/15] Step 220/300 | Loss=0.085788 | PredAvg=117.5 | TargetAvg=119.6
| MAE=28.41

[Warmstart 7/15] Step 240/300 | Loss=0.098114 | PredAvg=117.4 | TargetAvg=121.2
| MAE=29.60

[Warmstart 7/15] Step 260/300 | Loss=0.092947 | PredAvg=118.0 | TargetAvg=120.0
| MAE=29.30

[Warmstart 7/15] Step 280/300 | Loss=0.095805 | PredAvg=116.7 | TargetAvg=120.3

| MAE=29.34
[Warmstart 7/15] Step 300/300 | Loss=0.096004 | PredAvg=116.5 | TargetAvg=120.0
| MAE=29.00
Warmstart epoch 7 summary | Loss=0.096004 | PredAvg=116.5 | TargetAvg=120.0

Warm-start deterministic evaluation on test set...

Det eval	10/100	Acc=0.5000	AvgBudget=113.20	StdBudget=12.41
Det eval	20/100	Acc=0.4000	AvgBudget=113.25	StdBudget=10.05
Det eval	30/100	Acc=0.3667	AvgBudget=113.97	StdBudget=8.74
Det eval	40/100	Acc=0.3250	AvgBudget=114.33	StdBudget=8.24
Det eval	50/100	Acc=0.3400	AvgBudget=113.74	StdBudget=9.32
Det eval	60/100	Acc=0.3500	AvgBudget=113.58	StdBudget=9.43
Det eval	70/100	Acc=0.3571	AvgBudget=113.90	StdBudget=9.08
Det eval	80/100	Acc=0.3500	AvgBudget=113.99	StdBudget=9.19
Det eval	90/100	Acc=0.3333	AvgBudget=114.18	StdBudget=9.48
Det eval	100/100	Acc=0.3300	AvgBudget=113.98	StdBudget=9.44

Saved best warm-start policy by reward = 0.0706

[Warmstart 8/15] Step 20/300 | Loss=0.101304 | PredAvg=108.3 | TargetAvg=111.4 |
MAE=22.70
[Warmstart 8/15] Step 40/300 | Loss=0.100426 | PredAvg=105.3 | TargetAvg=113.5 |
MAE=22.05
[Warmstart 8/15] Step 60/300 | Loss=0.089934 | PredAvg=108.8 | TargetAvg=115.0 |
MAE=23.33
[Warmstart 8/15] Step 80/300 | Loss=0.100691 | PredAvg=111.2 | TargetAvg=120.5 |
MAE=25.60
[Warmstart 8/15] Step 100/300 | Loss=0.100563 | PredAvg=115.7 | TargetAvg=123.2
| MAE=27.72
[Warmstart 8/15] Step 120/300 | Loss=0.092209 | PredAvg=117.5 | TargetAvg=121.6
| MAE=27.63
[Warmstart 8/15] Step 140/300 | Loss=0.083717 | PredAvg=114.1 | TargetAvg=119.3
| MAE=26.18
[Warmstart 8/15] Step 160/300 | Loss=0.090109 | PredAvg=114.8 | TargetAvg=120.8
| MAE=27.04
[Warmstart 8/15] Step 180/300 | Loss=0.093610 | PredAvg=116.2 | TargetAvg=121.2
| MAE=28.12
[Warmstart 8/15] Step 200/300 | Loss=0.095773 | PredAvg=114.7 | TargetAvg=121.1
| MAE=27.89
[Warmstart 8/15] Step 220/300 | Loss=0.090711 | PredAvg=115.0 | TargetAvg=120.4
| MAE=27.35
[Warmstart 8/15] Step 240/300 | Loss=0.094182 | PredAvg=114.4 | TargetAvg=120.4
| MAE=27.63
[Warmstart 8/15] Step 260/300 | Loss=0.089731 | PredAvg=115.0 | TargetAvg=120.1
| MAE=27.29
[Warmstart 8/15] Step 280/300 | Loss=0.088346 | PredAvg=114.9 | TargetAvg=119.8
| MAE=27.13
[Warmstart 8/15] Step 300/300 | Loss=0.089264 | PredAvg=114.5 | TargetAvg=120.0
| MAE=27.14

Warmstart epoch 8 summary | Loss=0.089264 | PredAvg=114.5 | TargetAvg=120.0

Warm-start deterministic evaluation on test set...

Det eval	10/100	Acc=0.3000	AvgBudget=112.20	StdBudget=10.67
Det eval	20/100	Acc=0.2500	AvgBudget=112.40	StdBudget=8.74
Det eval	30/100	Acc=0.2667	AvgBudget=112.67	StdBudget=7.73
Det eval	40/100	Acc=0.2500	AvgBudget=112.30	StdBudget=7.24
Det eval	50/100	Acc=0.2800	AvgBudget=112.18	StdBudget=8.01
Det eval	60/100	Acc=0.3167	AvgBudget=112.05	StdBudget=7.91
Det eval	70/100	Acc=0.3143	AvgBudget=112.44	StdBudget=7.64
Det eval	80/100	Acc=0.3000	AvgBudget=112.62	StdBudget=7.74
Det eval	90/100	Acc=0.2889	AvgBudget=112.70	StdBudget=7.85
Det eval	100/100	Acc=0.2900	AvgBudget=112.55	StdBudget=7.73

[Warmstart 9/15] Step 20/300 | Loss=0.062756 | PredAvg=106.5 | TargetAvg=112.4 | MAE=19.35

[Warmstart 9/15] Step 40/300 | Loss=0.078531 | PredAvg=108.5 | TargetAvg=116.7 | MAE=20.50

[Warmstart 9/15] Step 60/300 | Loss=0.079433 | PredAvg=112.1 | TargetAvg=117.3 | MAE=22.18

[Warmstart 9/15] Step 80/300 | Loss=0.087142 | PredAvg=110.8 | TargetAvg=117.4 | MAE=22.89

[Warmstart 9/15] Step 100/300 | Loss=0.088929 | PredAvg=110.3 | TargetAvg=118.1 | MAE=23.69

[Warmstart 9/15] Step 120/300 | Loss=0.094246 | PredAvg=111.7 | TargetAvg=119.5 | MAE=25.44

[Warmstart 9/15] Step 140/300 | Loss=0.092463 | PredAvg=111.8 | TargetAvg=119.5 | MAE=25.36

[Warmstart 9/15] Step 160/300 | Loss=0.101456 | PredAvg=110.5 | TargetAvg=119.6 | MAE=25.59

[Warmstart 9/15] Step 180/300 | Loss=0.100254 | PredAvg=110.1 | TargetAvg=119.0 | MAE=25.10

[Warmstart 9/15] Step 200/300 | Loss=0.099368 | PredAvg=110.4 | TargetAvg=119.3 | MAE=25.59

[Warmstart 9/15] Step 220/300 | Loss=0.092587 | PredAvg=109.7 | TargetAvg=118.1 | MAE=24.50

[Warmstart 9/15] Step 240/300 | Loss=0.092969 | PredAvg=110.0 | TargetAvg=118.9 | MAE=24.58

[Warmstart 9/15] Step 260/300 | Loss=0.089252 | PredAvg=111.7 | TargetAvg=118.9 | MAE=24.82

[Warmstart 9/15] Step 280/300 | Loss=0.091071 | PredAvg=112.1 | TargetAvg=119.8 | MAE=25.27

[Warmstart 9/15] Step 300/300 | Loss=0.088327 | PredAvg=113.5 | TargetAvg=120.0 | MAE=25.87

Warmstart epoch 9 summary | Loss=0.088327 | PredAvg=113.5 | TargetAvg=120.0

Warm-start deterministic evaluation on test set...

Det eval	10/100	Acc=0.4000	AvgBudget=127.90	StdBudget=9.68
Det eval	20/100	Acc=0.3000	AvgBudget=128.25	StdBudget=7.71

Det eval	30/100	Acc=0.3000	AvgBudget=129.13	StdBudget=6.76
Det eval	40/100	Acc=0.2500	AvgBudget=129.38	StdBudget=6.42
Det eval	50/100	Acc=0.2600	AvgBudget=128.88	StdBudget=7.45
Det eval	60/100	Acc=0.2833	AvgBudget=128.82	StdBudget=7.55
Det eval	70/100	Acc=0.3000	AvgBudget=129.06	StdBudget=7.31
Det eval	80/100	Acc=0.2875	AvgBudget=129.16	StdBudget=7.40
Det eval	90/100	Acc=0.3000	AvgBudget=129.23	StdBudget=7.56
Det eval	100/100	Acc=0.3200	AvgBudget=129.10	StdBudget=7.55
[Warmstart 10/15] Step 20/300 Loss=0.072064 PredAvg=116.3 TargetAvg=117.2 MAE=28.35				
[Warmstart 10/15] Step 40/300 Loss=0.054151 PredAvg=118.1 TargetAvg=115.8 MAE=25.70				
[Warmstart 10/15] Step 60/300 Loss=0.056157 PredAvg=118.1 TargetAvg=117.9 MAE=25.33				
[Warmstart 10/15] Step 80/300 Loss=0.062489 PredAvg=119.5 TargetAvg=119.6 MAE=27.57				
[Warmstart 10/15] Step 100/300 Loss=0.070543 PredAvg=120.0 TargetAvg=121.3 MAE=28.97				
[Warmstart 10/15] Step 120/300 Loss=0.075483 PredAvg=119.7 TargetAvg=120.8 MAE=29.07				
[Warmstart 10/15] Step 140/300 Loss=0.077157 PredAvg=119.7 TargetAvg=121.0 MAE=29.36				
[Warmstart 10/15] Step 160/300 Loss=0.085697 PredAvg=117.7 TargetAvg=121.5 MAE=29.18				
[Warmstart 10/15] Step 180/300 Loss=0.090106 PredAvg=118.0 TargetAvg=122.2 MAE=29.71				
[Warmstart 10/15] Step 200/300 Loss=0.085690 PredAvg=116.9 TargetAvg=121.1 MAE=28.59				
[Warmstart 10/15] Step 220/300 Loss=0.082132 PredAvg=116.2 TargetAvg=120.2 MAE=27.75				
[Warmstart 10/15] Step 240/300 Loss=0.085234 PredAvg=115.9 TargetAvg=120.3 MAE=27.84				
[Warmstart 10/15] Step 260/300 Loss=0.082338 PredAvg=116.0 TargetAvg=119.7 MAE=27.50				
[Warmstart 10/15] Step 280/300 Loss=0.087125 PredAvg=115.1 TargetAvg=120.2 MAE=27.51				
[Warmstart 10/15] Step 300/300 Loss=0.086306 PredAvg=115.5 TargetAvg=120.0 MAE=27.50				
Warmstart epoch 10 summary Loss=0.086306 PredAvg=115.5 TargetAvg=120.0				

Warm-start deterministic evaluation on test set...

Det eval	10/100	Acc=0.4000	AvgBudget=117.20	StdBudget=9.45
Det eval	20/100	Acc=0.3000	AvgBudget=117.90	StdBudget=8.17
Det eval	30/100	Acc=0.3000	AvgBudget=117.87	StdBudget=7.78
Det eval	40/100	Acc=0.3000	AvgBudget=116.45	StdBudget=7.73
Det eval	50/100	Acc=0.3200	AvgBudget=116.86	StdBudget=7.79
Det eval	60/100	Acc=0.3333	AvgBudget=116.72	StdBudget=7.50
Det eval	70/100	Acc=0.3429	AvgBudget=117.20	StdBudget=7.27

Det eval | 80/100 | Acc=0.3250 | AvgBudget=117.49 | StdBudget=7.21
 Det eval | 90/100 | Acc=0.3222 | AvgBudget=117.34 | StdBudget=7.15
 Det eval | 100/100 | Acc=0.3200 | AvgBudget=117.19 | StdBudget=6.94
 [Warmstart 11/15] Step 20/300 | Loss=0.048570 | PredAvg=99.4 | TargetAvg=104.8 |
 MAE=12.70
 [Warmstart 11/15] Step 40/300 | Loss=0.095254 | PredAvg=106.2 | TargetAvg=116.3
 | MAE=21.52
 [Warmstart 11/15] Step 60/300 | Loss=0.074464 | PredAvg=111.7 | TargetAvg=117.4
 | MAE=22.52
 [Warmstart 11/15] Step 80/300 | Loss=0.071564 | PredAvg=110.3 | TargetAvg=116.5
 | MAE=21.91
 [Warmstart 11/15] Step 100/300 | Loss=0.077804 | PredAvg=110.5 | TargetAvg=117.6
 | MAE=22.98
 [Warmstart 11/15] Step 120/300 | Loss=0.080166 | PredAvg=112.7 | TargetAvg=118.5
 | MAE=24.05
 [Warmstart 11/15] Step 140/300 | Loss=0.083119 | PredAvg=113.3 | TargetAvg=118.5
 | MAE=24.40
 [Warmstart 11/15] Step 160/300 | Loss=0.086288 | PredAvg=112.0 | TargetAvg=118.8
 | MAE=24.36
 [Warmstart 11/15] Step 180/300 | Loss=0.090682 | PredAvg=112.5 | TargetAvg=119.4
 | MAE=25.02
 [Warmstart 11/15] Step 200/300 | Loss=0.089496 | PredAvg=111.9 | TargetAvg=119.1
 | MAE=24.77
 [Warmstart 11/15] Step 220/300 | Loss=0.086328 | PredAvg=112.8 | TargetAvg=119.0
 | MAE=25.33
 [Warmstart 11/15] Step 240/300 | Loss=0.087460 | PredAvg=114.2 | TargetAvg=120.2
 | MAE=26.00
 [Warmstart 11/15] Step 260/300 | Loss=0.084409 | PredAvg=113.9 | TargetAvg=119.4
 | MAE=25.58
 [Warmstart 11/15] Step 280/300 | Loss=0.082800 | PredAvg=113.7 | TargetAvg=119.4
 | MAE=25.46
 [Warmstart 11/15] Step 300/300 | Loss=0.083637 | PredAvg=114.4 | TargetAvg=120.0
 | MAE=25.87
 Warmstart epoch 11 summary | Loss=0.083637 | PredAvg=114.4 | TargetAvg=120.0

Warm-start deterministic evaluation on test set...

Det eval | 10/100 | Acc=0.5000 | AvgBudget=116.30 | StdBudget=10.04
 Det eval | 20/100 | Acc=0.3500 | AvgBudget=116.75 | StdBudget=9.21
 Det eval | 30/100 | Acc=0.3333 | AvgBudget=116.27 | StdBudget=9.41
 Det eval | 40/100 | Acc=0.3250 | AvgBudget=114.38 | StdBudget=9.36
 Det eval | 50/100 | Acc=0.3200 | AvgBudget=115.40 | StdBudget=9.33
 Det eval | 60/100 | Acc=0.3333 | AvgBudget=115.25 | StdBudget=8.91
 Det eval | 70/100 | Acc=0.3286 | AvgBudget=115.73 | StdBudget=8.67
 Det eval | 80/100 | Acc=0.3125 | AvgBudget=115.97 | StdBudget=8.37
 Det eval | 90/100 | Acc=0.3111 | AvgBudget=115.73 | StdBudget=8.25
 Det eval | 100/100 | Acc=0.3100 | AvgBudget=115.63 | StdBudget=8.05
 [Warmstart 12/15] Step 20/300 | Loss=0.079617 | PredAvg=121.0 | TargetAvg=125.6
 | MAE=29.50

[Warmstart 12/15] Step 40/300 | Loss=0.074648 | PredAvg=119.2 | TargetAvg=122.7
 | MAE=27.07
 [Warmstart 12/15] Step 60/300 | Loss=0.061855 | PredAvg=118.5 | TargetAvg=120.7
 | MAE=25.88
 [Warmstart 12/15] Step 80/300 | Loss=0.070579 | PredAvg=116.5 | TargetAvg=121.0
 | MAE=26.36
 [Warmstart 12/15] Step 100/300 | Loss=0.070153 | PredAvg=115.8 | TargetAvg=120.7
 | MAE=26.53
 [Warmstart 12/15] Step 120/300 | Loss=0.072206 | PredAvg=113.9 | TargetAvg=119.7
 | MAE=25.52
 [Warmstart 12/15] Step 140/300 | Loss=0.065599 | PredAvg=112.3 | TargetAvg=117.3
 | MAE=23.77
 [Warmstart 12/15] Step 160/300 | Loss=0.074869 | PredAvg=111.4 | TargetAvg=118.3
 | MAE=24.38
 [Warmstart 12/15] Step 180/300 | Loss=0.084076 | PredAvg=112.7 | TargetAvg=119.6
 | MAE=25.64
 [Warmstart 12/15] Step 200/300 | Loss=0.081780 | PredAvg=111.9 | TargetAvg=119.5
 | MAE=25.46
 [Warmstart 12/15] Step 220/300 | Loss=0.077885 | PredAvg=111.9 | TargetAvg=118.5
 | MAE=24.84
 [Warmstart 12/15] Step 240/300 | Loss=0.084480 | PredAvg=111.9 | TargetAvg=119.3
 | MAE=25.49
 [Warmstart 12/15] Step 260/300 | Loss=0.084487 | PredAvg=111.9 | TargetAvg=119.7
 | MAE=25.35
 [Warmstart 12/15] Step 280/300 | Loss=0.082238 | PredAvg=112.4 | TargetAvg=119.2
 | MAE=25.24
 [Warmstart 12/15] Step 300/300 | Loss=0.088795 | PredAvg=111.6 | TargetAvg=120.0
 | MAE=25.89
 Warmstart epoch 12 summary | Loss=0.088795 | PredAvg=111.6 | TargetAvg=120.0

Warm-start deterministic evaluation on test set...

Det eval | 10/100 | Acc=0.4000 | AvgBudget=118.40 | StdBudget=10.23
 Det eval | 20/100 | Acc=0.3000 | AvgBudget=118.75 | StdBudget=8.81
 Det eval | 30/100 | Acc=0.3000 | AvgBudget=118.57 | StdBudget=8.87
 Det eval | 40/100 | Acc=0.3000 | AvgBudget=116.53 | StdBudget=9.29
 Det eval | 50/100 | Acc=0.3200 | AvgBudget=117.42 | StdBudget=9.09
 Det eval | 60/100 | Acc=0.3333 | AvgBudget=117.18 | StdBudget=8.76
 Det eval | 70/100 | Acc=0.3429 | AvgBudget=117.69 | StdBudget=8.47
 Det eval | 80/100 | Acc=0.3250 | AvgBudget=118.05 | StdBudget=8.27
 Det eval | 90/100 | Acc=0.3111 | AvgBudget=117.68 | StdBudget=8.29
 Det eval | 100/100 | Acc=0.3100 | AvgBudget=117.53 | StdBudget=8.09
 [Warmstart 13/15] Step 20/300 | Loss=0.063671 | PredAvg=127.6 | TargetAvg=122.8
 | MAE=29.00
 [Warmstart 13/15] Step 40/300 | Loss=0.088031 | PredAvg=119.0 | TargetAvg=120.8
 | MAE=27.45
 [Warmstart 13/15] Step 60/300 | Loss=0.072654 | PredAvg=116.1 | TargetAvg=119.0
 | MAE=25.73
 [Warmstart 13/15] Step 80/300 | Loss=0.066613 | PredAvg=117.1 | TargetAvg=117.8

| MAE=25.26
 [Warmstart 13/15] Step 100/300 | Loss=0.087451 | PredAvg=116.7 | TargetAvg=121.7
 | MAE=27.54
 [Warmstart 13/15] Step 120/300 | Loss=0.085054 | PredAvg=118.6 | TargetAvg=121.5
 | MAE=28.10
 [Warmstart 13/15] Step 140/300 | Loss=0.085340 | PredAvg=116.7 | TargetAvg=120.0
 | MAE=27.15
 [Warmstart 13/15] Step 160/300 | Loss=0.083880 | PredAvg=115.1 | TargetAvg=119.6
 | MAE=26.44
 [Warmstart 13/15] Step 180/300 | Loss=0.088342 | PredAvg=114.6 | TargetAvg=119.5
 | MAE=26.34
 [Warmstart 13/15] Step 200/300 | Loss=0.083539 | PredAvg=115.0 | TargetAvg=119.3
 | MAE=26.27
 [Warmstart 13/15] Step 220/300 | Loss=0.078428 | PredAvg=114.6 | TargetAvg=118.6
 | MAE=25.48
 [Warmstart 13/15] Step 240/300 | Loss=0.082972 | PredAvg=114.8 | TargetAvg=120.2
 | MAE=26.21
 [Warmstart 13/15] Step 260/300 | Loss=0.082121 | PredAvg=115.5 | TargetAvg=120.5
 | MAE=26.21
 [Warmstart 13/15] Step 280/300 | Loss=0.083505 | PredAvg=115.4 | TargetAvg=121.0
 | MAE=26.35
 [Warmstart 13/15] Step 300/300 | Loss=0.079999 | PredAvg=114.8 | TargetAvg=120.0
 | MAE=25.82

Warmstart epoch 13 summary | Loss=0.079999 | PredAvg=114.8 | TargetAvg=120.0

Warm-start deterministic evaluation on test set...

Det eval | 10/100 | Acc=0.3000 | AvgBudget=101.30 | StdBudget=10.37
 Det eval | 20/100 | Acc=0.2500 | AvgBudget=101.10 | StdBudget=9.11
 Det eval | 30/100 | Acc=0.2667 | AvgBudget=101.00 | StdBudget=9.22
 Det eval | 40/100 | Acc=0.2750 | AvgBudget=99.65 | StdBudget=8.76
 Det eval | 50/100 | Acc=0.3000 | AvgBudget=100.44 | StdBudget=9.08
 Det eval | 60/100 | Acc=0.3167 | AvgBudget=100.23 | StdBudget=8.66
 Det eval | 70/100 | Acc=0.3000 | AvgBudget=100.60 | StdBudget=8.39
 Det eval | 80/100 | Acc=0.2875 | AvgBudget=100.75 | StdBudget=8.15
 Det eval | 90/100 | Acc=0.3000 | AvgBudget=100.59 | StdBudget=8.29
 Det eval | 100/100 | Acc=0.2900 | AvgBudget=100.37 | StdBudget=8.09
 [Warmstart 14/15] Step 20/300 | Loss=0.041703 | PredAvg=104.6 | TargetAvg=109.6
 | MAE=18.20
 [Warmstart 14/15] Step 40/300 | Loss=0.087014 | PredAvg=108.7 | TargetAvg=117.3
 | MAE=23.93
 [Warmstart 14/15] Step 60/300 | Loss=0.072662 | PredAvg=113.2 | TargetAvg=118.3
 | MAE=24.27
 [Warmstart 14/15] Step 80/300 | Loss=0.069508 | PredAvg=112.7 | TargetAvg=117.7
 | MAE=23.62
 [Warmstart 14/15] Step 100/300 | Loss=0.073320 | PredAvg=112.7 | TargetAvg=118.7
 | MAE=23.85
 [Warmstart 14/15] Step 120/300 | Loss=0.068883 | PredAvg=112.1 | TargetAvg=117.2
 | MAE=22.84

[Warmstart 14/15] Step 140/300 | Loss=0.076972 | PredAvg=111.5 | TargetAvg=119.1
 | MAE=24.55
 [Warmstart 14/15] Step 160/300 | Loss=0.078011 | PredAvg=113.2 | TargetAvg=118.9
 | MAE=25.58
 [Warmstart 14/15] Step 180/300 | Loss=0.075955 | PredAvg=113.0 | TargetAvg=118.7
 | MAE=25.61
 [Warmstart 14/15] Step 200/300 | Loss=0.081300 | PredAvg=113.5 | TargetAvg=119.5
 | MAE=25.98
 [Warmstart 14/15] Step 220/300 | Loss=0.079329 | PredAvg=113.1 | TargetAvg=119.2
 | MAE=25.40
 [Warmstart 14/15] Step 240/300 | Loss=0.081594 | PredAvg=112.6 | TargetAvg=119.7
 | MAE=25.42
 [Warmstart 14/15] Step 260/300 | Loss=0.082350 | PredAvg=114.1 | TargetAvg=120.3
 | MAE=26.24
 [Warmstart 14/15] Step 280/300 | Loss=0.086127 | PredAvg=113.8 | TargetAvg=120.8
 | MAE=26.46
 [Warmstart 14/15] Step 300/300 | Loss=0.081480 | PredAvg=113.1 | TargetAvg=120.0
 | MAE=25.54
 Warmstart epoch 14 summary | Loss=0.081480 | PredAvg=113.1 | TargetAvg=120.0

Warm-start deterministic evaluation on test set...

Det eval | 10/100 | Acc=0.4000 | AvgBudget=113.20 | StdBudget=10.47
 Det eval | 20/100 | Acc=0.3000 | AvgBudget=113.50 | StdBudget=9.54
 Det eval | 30/100 | Acc=0.3000 | AvgBudget=113.17 | StdBudget=10.12
 Det eval | 40/100 | Acc=0.2750 | AvgBudget=111.17 | StdBudget=10.13
 Det eval | 50/100 | Acc=0.3000 | AvgBudget=112.16 | StdBudget=10.08
 Det eval | 60/100 | Acc=0.3167 | AvgBudget=111.93 | StdBudget=9.66
 Det eval | 70/100 | Acc=0.3000 | AvgBudget=112.41 | StdBudget=9.40
 Det eval | 80/100 | Acc=0.2875 | AvgBudget=112.70 | StdBudget=9.05
 Det eval | 90/100 | Acc=0.2778 | AvgBudget=112.40 | StdBudget=9.08
 Det eval | 100/100 | Acc=0.2700 | AvgBudget=112.23 | StdBudget=8.90
 [Warmstart 15/15] Step 20/300 | Loss=0.037567 | PredAvg=109.3 | TargetAvg=107.6
 | MAE=16.45
 [Warmstart 15/15] Step 40/300 | Loss=0.076831 | PredAvg=110.3 | TargetAvg=118.6
 | MAE=22.05
 [Warmstart 15/15] Step 60/300 | Loss=0.069052 | PredAvg=115.8 | TargetAvg=120.7
 | MAE=25.05
 [Warmstart 15/15] Step 80/300 | Loss=0.063673 | PredAvg=115.6 | TargetAvg=119.8
 | MAE=24.44
 [Warmstart 15/15] Step 100/300 | Loss=0.073651 | PredAvg=114.5 | TargetAvg=121.6
 | MAE=24.88
 [Warmstart 15/15] Step 120/300 | Loss=0.071925 | PredAvg=118.5 | TargetAvg=121.4
 | MAE=27.26
 [Warmstart 15/15] Step 140/300 | Loss=0.071946 | PredAvg=120.6 | TargetAvg=121.9
 | MAE=27.91
 [Warmstart 15/15] Step 160/300 | Loss=0.072396 | PredAvg=119.2 | TargetAvg=121.1
 | MAE=26.96
 [Warmstart 15/15] Step 180/300 | Loss=0.072242 | PredAvg=118.7 | TargetAvg=121.5

```

| MAE=26.42
[Warmstart 15/15] Step 200/300 | Loss=0.074568 | PredAvg=118.2 | TargetAvg=120.7
| MAE=26.32
[Warmstart 15/15] Step 220/300 | Loss=0.083106 | PredAvg=116.6 | TargetAvg=121.0
| MAE=26.50
[Warmstart 15/15] Step 240/300 | Loss=0.081318 | PredAvg=116.8 | TargetAvg=120.6
| MAE=26.53
[Warmstart 15/15] Step 260/300 | Loss=0.077232 | PredAvg=116.7 | TargetAvg=119.9
| MAE=25.95
[Warmstart 15/15] Step 280/300 | Loss=0.083627 | PredAvg=116.1 | TargetAvg=120.5
| MAE=26.23
[Warmstart 15/15] Step 300/300 | Loss=0.081998 | PredAvg=115.6 | TargetAvg=120.0
| MAE=25.84
Warmstart epoch 15 summary | Loss=0.081998 | PredAvg=115.6 | TargetAvg=120.0

```

Warm-start deterministic evaluation on test set...

```

Det eval | 10/100 | Acc=0.3000 | AvgBudget=109.70 | StdBudget=11.00
Det eval | 20/100 | Acc=0.2000 | AvgBudget=110.35 | StdBudget=10.48
Det eval | 30/100 | Acc=0.2333 | AvgBudget=109.93 | StdBudget=11.45
Det eval | 40/100 | Acc=0.2000 | AvgBudget=107.72 | StdBudget=11.42
Det eval | 50/100 | Acc=0.2000 | AvgBudget=108.84 | StdBudget=11.32
Det eval | 60/100 | Acc=0.2333 | AvgBudget=108.53 | StdBudget=10.89
Det eval | 70/100 | Acc=0.2429 | AvgBudget=109.09 | StdBudget=10.65
Det eval | 80/100 | Acc=0.2375 | AvgBudget=109.41 | StdBudget=10.24
Det eval | 90/100 | Acc=0.2444 | AvgBudget=109.18 | StdBudget=10.23
Det eval | 100/100 | Acc=0.2400 | AvgBudget=109.05 | StdBudget=9.99

```

Loading best warm-start policy by accuracy...

Evaluating deterministic policy right after warm-start...

```

Det eval | 10/100 | Acc=0.4000 | AvgBudget=136.60 | StdBudget=8.15
Det eval | 20/100 | Acc=0.3000 | AvgBudget=136.70 | StdBudget=6.58
Det eval | 30/100 | Acc=0.3000 | AvgBudget=137.13 | StdBudget=5.68
Det eval | 40/100 | Acc=0.2750 | AvgBudget=137.40 | StdBudget=5.40
Det eval | 50/100 | Acc=0.2800 | AvgBudget=137.02 | StdBudget=6.13
Det eval | 60/100 | Acc=0.3333 | AvgBudget=136.95 | StdBudget=6.20
Det eval | 70/100 | Acc=0.3429 | AvgBudget=137.17 | StdBudget=6.00
Det eval | 80/100 | Acc=0.3375 | AvgBudget=137.20 | StdBudget=6.12
Det eval | 90/100 | Acc=0.3333 | AvgBudget=137.29 | StdBudget=6.27
Det eval | 100/100 | Acc=0.3600 | AvgBudget=137.19 | StdBudget=6.27

```

```

{
  "warmstart_deterministic_eval": {
    "accuracy": 0.36,
    "avg_thinking_tokens": 137.19,
    "avg_reward": 0.0384299999999999985,
    "avg_budget": 137.19,
    "min_budget": 121.0,
    "max_budget": 151.0,
    "std_budget": 6.273268685462149
  }
}

```

```

}
}

```

Evaluating sampled policy right after warm-start...

```

Samp eval | 10/100 | Acc=0.3000 | AvgBudget=141.60
Samp eval | 20/100 | Acc=0.2500 | AvgBudget=145.65
Samp eval | 30/100 | Acc=0.3333 | AvgBudget=138.47
Samp eval | 40/100 | Acc=0.3250 | AvgBudget=139.43
Samp eval | 50/100 | Acc=0.3600 | AvgBudget=135.82
Samp eval | 60/100 | Acc=0.4000 | AvgBudget=137.08
Samp eval | 70/100 | Acc=0.4143 | AvgBudget=136.84
Samp eval | 80/100 | Acc=0.4125 | AvgBudget=138.66
Samp eval | 90/100 | Acc=0.4111 | AvgBudget=141.72
Samp eval | 100/100 | Acc=0.4200 | AvgBudget=142.30
{
  "warmstart_sampled_eval": {
    "accuracy": 0.42,
    "avg_thinking_tokens": 142.3,
    "avg_reward": 0.0981,
    "avg_budget": 142.3,
    "min_budget": 67.0,
    "max_budget": 215.0,
    "std_budget": 43.98238283676773
  }
}

```

Starting RL fine-tuning...

```

[RL 1/8] Step 20/300 | Reward=0.1714 | Acc=0.5000 | AvgBudget=151.2 |
StdBudget=35.4
  Recent budgets: [191, 117, 179, 136, 134, 139, 186, 88, 82, 84]
[RL 1/8] Step 40/300 | Reward=0.0366 | Acc=0.3750 | AvgBudget=144.1 |
StdBudget=38.5
  Recent budgets: [194, 171, 94, 212, 171, 142, 97, 149, 94, 175]
[RL 1/8] Step 60/300 | Reward=0.0111 | Acc=0.3500 | AvgBudget=142.1 |
StdBudget=39.6
  Recent budgets: [209, 150, 151, 128, 157, 69, 72, 191, 134, 67]
[RL 1/8] Step 80/300 | Reward=-0.0346 | Acc=0.3125 | AvgBudget=141.8 |
StdBudget=41.6
  Recent budgets: [70, 79, 111, 199, 160, 171, 154, 125, 180, 211]
[RL 1/8] Step 100/300 | Reward=0.0080 | Acc=0.3400 | AvgBudget=139.0 |
StdBudget=42.4
  Recent budgets: [92, 218, 164, 81, 76, 71, 121, 173, 193, 114]
[RL 1/8] Step 120/300 | Reward=-0.0048 | Acc=0.3250 | AvgBudget=137.0 |
StdBudget=42.6
  Recent budgets: [148, 167, 76, 135, 101, 197, 185, 214, 70, 94]
[RL 1/8] Step 140/300 | Reward=-0.0097 | Acc=0.3214 | AvgBudget=137.2 |
StdBudget=43.4
  Recent budgets: [160, 184, 188, 181, 113, 189, 117, 73, 164, 84]

```

[RL 1/8] Step 160/300 | Reward=-0.0183 | Acc=0.3125 | AvgBudget=136.3 | StdBudget=43.1

Recent budgets: [187, 100, 147, 183, 113, 82, 75, 76, 85, 117]

[RL 1/8] Step 180/300 | Reward=-0.0081 | Acc=0.3222 | AvgBudget=137.0 | StdBudget=44.0

Recent budgets: [93, 111, 207, 130, 96, 99, 136, 213, 163, 210]

[RL 1/8] Step 200/300 | Reward=-0.0024 | Acc=0.3300 | AvgBudget=138.3 | StdBudget=44.3

Recent budgets: [106, 217, 166, 180, 211, 193, 140, 196, 94, 73]

[RL 1/8] Step 220/300 | Reward=0.0020 | Acc=0.3364 | AvgBudget=139.5 | StdBudget=44.6

Recent budgets: [215, 186, 189, 137, 160, 71, 102, 161, 190, 176]

[RL 1/8] Step 240/300 | Reward=0.0071 | Acc=0.3375 | AvgBudget=138.2 | StdBudget=44.8

Recent budgets: [215, 102, 184, 105, 76, 148, 79, 178, 93, 157]

[RL 1/8] Step 260/300 | Reward=0.0078 | Acc=0.3385 | AvgBudget=138.4 | StdBudget=45.3

Recent budgets: [67, 71, 154, 131, 87, 92, 142, 81, 195, 156]

[RL 1/8] Step 280/300 | Reward=0.0111 | Acc=0.3393 | AvgBudget=137.7 | StdBudget=45.0

Recent budgets: [153, 150, 111, 131, 211, 164, 100, 153, 157, 130]

[RL 1/8] Step 300/300 | Reward=0.0226 | Acc=0.3467 | AvgBudget=136.9 | StdBudget=44.6

Recent budgets: [105, 174, 170, 112, 162, 112, 132, 146, 205, 163]

=====

RL Epoch 1 summary

Train Accuracy : 0.3467

Train Avg Tokens: 136.9

Train Avg Reward: 0.0226

Train Avg Budget: 136.9 +/- 44.6

Deterministic policy evaluation on test set...

Det eval | 10/100 | Acc=0.4000 | AvgBudget=110.40 | StdBudget=11.50

Det eval | 20/100 | Acc=0.3500 | AvgBudget=110.20 | StdBudget=9.61

Det eval | 30/100 | Acc=0.3333 | AvgBudget=110.47 | StdBudget=8.46

Det eval | 40/100 | Acc=0.3000 | AvgBudget=110.47 | StdBudget=8.01

Det eval | 50/100 | Acc=0.3200 | AvgBudget=110.30 | StdBudget=8.70

Det eval | 60/100 | Acc=0.3333 | AvgBudget=110.20 | StdBudget=8.62

Det eval | 70/100 | Acc=0.3286 | AvgBudget=110.56 | StdBudget=8.30

Det eval | 80/100 | Acc=0.3125 | AvgBudget=110.58 | StdBudget=8.40

Det eval | 90/100 | Acc=0.2889 | AvgBudget=110.74 | StdBudget=8.67

Det eval | 100/100 | Acc=0.2900 | AvgBudget=110.59 | StdBudget=8.59

Test Accuracy : 0.2900

Test AvgTokens : 110.6

Test AvgReward : 0.0307

Test Budget : 110.6 +/- 8.6 [91, 134]

=====

Saved best RL models by reward = 0.0307

Saved best RL policy by accuracy = 0.2900

[RL 2/8] Step 20/300 | Reward=0.2181 | Acc=0.5500 | AvgBudget=156.4 | StdBudget=44.6

Recent budgets: [203, 212, 151, 98, 204, 145, 174, 87, 161, 195]

[RL 2/8] Step 40/300 | Reward=0.0934 | Acc=0.4250 | AvgBudget=145.9 | StdBudget=41.9

Recent budgets: [129, 94, 186, 79, 174, 178, 152, 90, 109, 150]

[RL 2/8] Step 60/300 | Reward=0.1659 | Acc=0.4667 | AvgBudget=139.2 | StdBudget=44.5

Recent budgets: [105, 69, 174, 73, 82, 90, 72, 110, 142, 176]

[RL 2/8] Step 80/300 | Reward=0.1860 | Acc=0.4875 | AvgBudget=141.1 | StdBudget=43.7

Recent budgets: [130, 192, 101, 168, 78, 104, 215, 118, 153, 158]

[RL 2/8] Step 100/300 | Reward=0.1291 | Acc=0.4400 | AvgBudget=140.3 | StdBudget=42.4

Recent budgets: [151, 143, 189, 204, 103, 101, 162, 142, 151, 87]

[RL 2/8] Step 120/300 | Reward=0.0819 | Acc=0.4000 | AvgBudget=139.4 | StdBudget=41.6

Recent budgets: [143, 157, 82, 125, 154, 109, 174, 146, 198, 162]

[RL 2/8] Step 140/300 | Reward=0.0506 | Acc=0.3714 | AvgBudget=137.9 | StdBudget=42.9

Recent budgets: [167, 203, 156, 77, 203, 92, 112, 147, 167, 181]

[RL 2/8] Step 160/300 | Reward=0.0254 | Acc=0.3438 | AvgBudget=134.8 | StdBudget=43.1

Recent budgets: [94, 160, 124, 75, 119, 107, 180, 65, 98, 130]

[RL 2/8] Step 180/300 | Reward=0.0152 | Acc=0.3333 | AvgBudget=133.8 | StdBudget=43.9

Recent budgets: [103, 84, 98, 72, 208, 76, 203, 118, 171, 93]

[RL 2/8] Step 200/300 | Reward=0.0206 | Acc=0.3350 | AvgBudget=132.7 | StdBudget=44.0

Recent budgets: [75, 105, 157, 213, 85, 130, 158, 83, 81, 105]

[RL 2/8] Step 220/300 | Reward=0.0423 | Acc=0.3545 | AvgBudget=133.6 | StdBudget=44.7

Recent budgets: [88, 164, 114, 195, 152, 212, 85, 69, 199, 97]

[RL 2/8] Step 240/300 | Reward=0.0268 | Acc=0.3375 | AvgBudget=131.7 | StdBudget=44.7

Recent budgets: [88, 88, 89, 72, 139, 68, 101, 166, 194, 110]

[RL 2/8] Step 260/300 | Reward=0.0265 | Acc=0.3385 | AvgBudget=132.2 | StdBudget=44.7

Recent budgets: [94, 199, 120, 219, 117, 66, 74, 182, 135, 131]

[RL 2/8] Step 280/300 | Reward=0.0340 | Acc=0.3464 | AvgBudget=133.0 | StdBudget=44.8

Recent budgets: [85, 207, 212, 107, 196, 141, 144, 164, 69, 147]
[RL 2/8] Step 300/300 | Reward=0.0296 | Acc=0.3433 | AvgBudget=133.2 | StdBudget=44.9

Recent budgets: [178, 188, 160, 66, 133, 219, 184, 132, 140, 99]

=====

RL Epoch 2 summary

Train Accuracy : 0.3433

Train Avg Tokens: 133.2

Train Avg Reward: 0.0296

Train Avg Budget: 133.2 +/- 44.9

Deterministic policy evaluation on test set...

Det eval | 10/100 | Acc=0.3000 | AvgBudget=108.50 | StdBudget=11.60

Det eval | 20/100 | Acc=0.3000 | AvgBudget=108.45 | StdBudget=9.57

Det eval | 30/100 | Acc=0.3000 | AvgBudget=108.57 | StdBudget=8.42

Det eval | 40/100 | Acc=0.3000 | AvgBudget=108.35 | StdBudget=7.87

Det eval | 50/100 | Acc=0.3200 | AvgBudget=108.26 | StdBudget=8.51

Det eval | 60/100 | Acc=0.3333 | AvgBudget=108.15 | StdBudget=8.39

Det eval | 70/100 | Acc=0.3286 | AvgBudget=108.61 | StdBudget=8.12

Det eval | 80/100 | Acc=0.3125 | AvgBudget=108.72 | StdBudget=8.21

Det eval | 90/100 | Acc=0.2889 | AvgBudget=108.89 | StdBudget=8.37

Det eval | 100/100 | Acc=0.2900 | AvgBudget=108.77 | StdBudget=8.25

Test Accuracy : 0.2900

Test AvgTokens : 108.8

Test AvgReward : 0.0362

Test Budget : 108.8 +/- 8.2 [91, 132]

=====

Saved best RL models by reward = 0.0362

[RL 3/8] Step 20/300 | Reward=-0.1152 | Acc=0.2500 | AvgBudget=142.6 | StdBudget=45.0

Recent budgets: [120, 165, 67, 171, 201, 116, 116, 179, 137, 152]

[RL 3/8] Step 40/300 | Reward=-0.1011 | Acc=0.2500 | AvgBudget=137.8 | StdBudget=46.2

Recent budgets: [117, 213, 77, 112, 172, 87, 167, 172, 76, 72]

[RL 3/8] Step 60/300 | Reward=-0.0978 | Acc=0.2500 | AvgBudget=136.8 | StdBudget=45.3

Recent budgets: [136, 142, 216, 88, 88, 94, 90, 123, 197, 111]

[RL 3/8] Step 80/300 | Reward=-0.0281 | Acc=0.3125 | AvgBudget=139.6 | StdBudget=45.5

Recent budgets: [103, 211, 114, 212, 212, 145, 165, 124, 85, 176]

[RL 3/8] Step 100/300 | Reward=-0.0434 | Acc=0.2900 | AvgBudget=135.3 | StdBudget=45.9

Recent budgets: [177, 90, 100, 76, 215, 80, 72, 192, 135, 67]

[RL 3/8] Step 120/300 | Reward=0.0130 | Acc=0.3417 | AvgBudget=138.0 | StdBudget=45.4
Recent budgets: [113, 99, 164, 146, 151, 192, 85, 170, 115, 211]
[RL 3/8] Step 140/300 | Reward=0.0220 | Acc=0.3429 | AvgBudget=135.5 | StdBudget=45.3
Recent budgets: [139, 121, 168, 113, 131, 195, 116, 98, 75, 180]
[RL 3/8] Step 160/300 | Reward=-0.0011 | Acc=0.3187 | AvgBudget=133.2 | StdBudget=44.5
Recent budgets: [131, 77, 100, 103, 158, 79, 84, 103, 110, 108]
[RL 3/8] Step 180/300 | Reward=0.0017 | Acc=0.3222 | AvgBudget=133.7 | StdBudget=44.6
Recent budgets: [188, 102, 82, 179, 194, 90, 184, 92, 81, 68]
[RL 3/8] Step 200/300 | Reward=-0.0017 | Acc=0.3200 | AvgBudget=133.9 | StdBudget=43.8
Recent budgets: [69, 74, 105, 137, 165, 86, 133, 174, 149, 125]
[RL 3/8] Step 220/300 | Reward=-0.0134 | Acc=0.3136 | AvgBudget=135.1 | StdBudget=44.5
Recent budgets: [101, 186, 88, 129, 186, 169, 200, 156, 71, 93]
[RL 3/8] Step 240/300 | Reward=-0.0041 | Acc=0.3250 | AvgBudget=136.8 | StdBudget=44.8
Recent budgets: [212, 153, 90, 190, 146, 219, 167, 187, 212, 94]
[RL 3/8] Step 260/300 | Reward=0.0058 | Acc=0.3346 | AvgBudget=137.5 | StdBudget=44.6
Recent budgets: [132, 68, 105, 198, 208, 112, 140, 146, 172, 132]
[RL 3/8] Step 280/300 | Reward=0.0152 | Acc=0.3429 | AvgBudget=137.8 | StdBudget=44.2
Recent budgets: [186, 110, 102, 166, 205, 172, 83, 70, 145, 188]
[RL 3/8] Step 300/300 | Reward=0.0258 | Acc=0.3567 | AvgBudget=140.0 | StdBudget=44.9
Recent budgets: [192, 205, 205, 71, 201, 181, 136, 200, 138, 149]

=====

RL Epoch 3 summary

Train Accuracy : 0.3567

Train Avg Tokens: 140.0

Train Avg Reward: 0.0258

Train Avg Budget: 140.0 +/- 44.9

Deterministic policy evaluation on test set...

Det eval	10/100	Acc=0.4000	AvgBudget=130.20	StdBudget=8.77
Det eval	20/100	Acc=0.3000	AvgBudget=130.55	StdBudget=7.00
Det eval	30/100	Acc=0.3000	AvgBudget=130.80	StdBudget=6.10
Det eval	40/100	Acc=0.2500	AvgBudget=129.93	StdBudget=6.10
Det eval	50/100	Acc=0.2800	AvgBudget=130.08	StdBudget=6.50
Det eval	60/100	Acc=0.3000	AvgBudget=130.03	StdBudget=6.34
Det eval	70/100	Acc=0.3143	AvgBudget=130.41	StdBudget=6.13
Det eval	80/100	Acc=0.3000	AvgBudget=130.60	StdBudget=6.21

```

    Det eval | 90/100 | Acc=0.2889 | AvgBudget=130.53 | StdBudget=6.16
    Det eval | 100/100 | Acc=0.3200 | AvgBudget=130.45 | StdBudget=6.04
Test Accuracy : 0.3200
Test AvgTokens : 130.4
Test AvgReward : 0.0086
Test Budget : 130.4 +/- 6.0 [117, 145]
=====
=====

```

Saved best RL policy by accuracy = 0.3200

```

[RL 4/8] Step 20/300 | Reward=-0.1324 | Acc=0.2500 | AvgBudget=148.3 |
StdBudget=45.4
    Recent budgets: [188, 155, 107, 70, 121, 88, 106, 205, 138, 179]
[RL 4/8] Step 40/300 | Reward=-0.0485 | Acc=0.2750 | AvgBudget=130.8 |
StdBudget=46.8
    Recent budgets: [76, 113, 73, 99, 71, 184, 113, 102, 99, 110]
[RL 4/8] Step 60/300 | Reward=0.0243 | Acc=0.3167 | AvgBudget=123.8 |
StdBudget=44.0
    Recent budgets: [127, 137, 214, 145, 66, 73, 102, 107, 69, 97]
[RL 4/8] Step 80/300 | Reward=0.0846 | Acc=0.3625 | AvgBudget=122.8 |
StdBudget=43.2
    Recent budgets: [91, 120, 110, 74, 68, 148, 70, 185, 146, 108]
[RL 4/8] Step 100/300 | Reward=0.1064 | Acc=0.3800 | AvgBudget=122.9 |
StdBudget=42.0
    Recent budgets: [142, 99, 142, 95, 108, 84, 140, 176, 121, 105]
[RL 4/8] Step 120/300 | Reward=0.0929 | Acc=0.3750 | AvgBudget=125.3 |
StdBudget=41.8
    Recent budgets: [104, 135, 69, 190, 167, 110, 198, 127, 172, 139]
[RL 4/8] Step 140/300 | Reward=0.0644 | Acc=0.3500 | AvgBudget=124.4 |
StdBudget=40.8
    Recent budgets: [124, 157, 96, 140, 79, 100, 74, 78, 168, 173]
[RL 4/8] Step 160/300 | Reward=0.0360 | Acc=0.3312 | AvgBudget=126.0 |
StdBudget=41.9
    Recent budgets: [82, 100, 166, 108, 163, 158, 154, 86, 201, 135]
[RL 4/8] Step 180/300 | Reward=0.0480 | Acc=0.3389 | AvgBudget=125.2 |
StdBudget=41.9
    Recent budgets: [77, 66, 99, 201, 109, 162, 196, 141, 141, 78]
[RL 4/8] Step 200/300 | Reward=0.0362 | Acc=0.3250 | AvgBudget=123.4 |
StdBudget=41.4
    Recent budgets: [170, 70, 102, 89, 119, 177, 111, 79, 131, 135]
[RL 4/8] Step 220/300 | Reward=0.0239 | Acc=0.3136 | AvgBudget=122.7 |
StdBudget=41.4
    Recent budgets: [169, 93, 114, 89, 109, 135, 82, 150, 191, 98]
[RL 4/8] Step 240/300 | Reward=0.0036 | Acc=0.2958 | AvgBudget=122.1 |
StdBudget=41.3
    Recent budgets: [150, 125, 87, 82, 71, 125, 74, 75, 102, 76]
[RL 4/8] Step 260/300 | Reward=0.0062 | Acc=0.3000 | AvgBudget=122.9 |

```

StdBudget=41.6

Recent budgets: [120, 212, 102, 83, 107, 71, 200, 105, 184, 180]

[RL 4/8] Step 280/300 | Reward=-0.0054 | Acc=0.2893 | AvgBudget=122.3 |

StdBudget=41.2

Recent budgets: [142, 82, 160, 117, 104, 146, 121, 89, 93, 107]

[RL 4/8] Step 300/300 | Reward=0.0056 | Acc=0.3000 | AvgBudget=123.1 |

StdBudget=41.4

Recent budgets: [92, 106, 88, 69, 193, 175, 198, 140, 103, 115]

=====

RL Epoch 4 summary

Train Accuracy : 0.3000

Train Avg Tokens: 123.1

Train Avg Reward: 0.0056

Train Avg Budget: 123.1 +/- 41.4

Deterministic policy evaluation on test set...

Det eval | 10/100 | Acc=0.3000 | AvgBudget=108.10 | StdBudget=10.66

Det eval | 20/100 | Acc=0.3000 | AvgBudget=108.10 | StdBudget=8.91

Det eval | 30/100 | Acc=0.3000 | AvgBudget=107.90 | StdBudget=8.27

Det eval | 40/100 | Acc=0.3250 | AvgBudget=107.00 | StdBudget=7.83

Det eval | 50/100 | Acc=0.3400 | AvgBudget=107.40 | StdBudget=8.11

Det eval | 60/100 | Acc=0.3667 | AvgBudget=107.32 | StdBudget=7.88

Det eval | 70/100 | Acc=0.3429 | AvgBudget=107.74 | StdBudget=7.66

Det eval | 80/100 | Acc=0.3250 | AvgBudget=107.85 | StdBudget=7.76

Det eval | 90/100 | Acc=0.3222 | AvgBudget=107.89 | StdBudget=7.80

Det eval | 100/100 | Acc=0.3200 | AvgBudget=107.79 | StdBudget=7.62

Test Accuracy : 0.3200

Test AvgTokens : 107.8

Test AvgReward : 0.0766

Test Budget : 107.8 +/- 7.6 [92, 129]

=====

Saved best RL models by reward = 0.0766

[RL 5/8] Step 20/300 | Reward=0.0883 | Acc=0.3500 | AvgBudget=116.4 |

StdBudget=42.3

Recent budgets: [66, 126, 125, 198, 132, 88, 152, 147, 89, 66]

[RL 5/8] Step 40/300 | Reward=-0.0345 | Acc=0.2500 | AvgBudget=115.7 |

StdBudget=40.8

Recent budgets: [172, 159, 103, 86, 116, 70, 93, 129, 71, 69]

[RL 5/8] Step 60/300 | Reward=0.0177 | Acc=0.2833 | AvgBudget=112.2 |

StdBudget=38.0

Recent budgets: [77, 104, 104, 167, 161, 69, 73, 94, 147, 138]

[RL 5/8] Step 80/300 | Reward=0.0424 | Acc=0.3000 | AvgBudget=110.8 |

StdBudget=40.2

Recent budgets: [79, 88, 202, 113, 142, 72, 100, 81, 75, 123]
 [RL 5/8] Step 100/300 | Reward=0.0084 | Acc=0.2800 | AvgBudget=113.9 | StdBudget=40.7

Recent budgets: [174, 91, 106, 92, 175, 95, 166, 154, 87, 188]
 [RL 5/8] Step 120/300 | Reward=-0.0222 | Acc=0.2583 | AvgBudget=115.0 | StdBudget=39.6

Recent budgets: [99, 135, 71, 107, 125, 89, 101, 138, 98, 107]
 [RL 5/8] Step 140/300 | Reward=-0.0226 | Acc=0.2571 | AvgBudget=114.7 | StdBudget=39.2

Recent budgets: [73, 70, 157, 76, 155, 121, 156, 72, 87, 72]
 [RL 5/8] Step 160/300 | Reward=-0.0064 | Acc=0.2750 | AvgBudget=116.7 | StdBudget=40.8

Recent budgets: [110, 165, 69, 151, 72, 87, 209, 204, 113, 193]
 [RL 5/8] Step 180/300 | Reward=-0.0269 | Acc=0.2611 | AvgBudget=117.8 | StdBudget=40.8

Recent budgets: [81, 96, 78, 165, 71, 81, 103, 170, 71, 134]
 [RL 5/8] Step 200/300 | Reward=-0.0249 | Acc=0.2650 | AvgBudget=118.7 | StdBudget=41.1

Recent budgets: [108, 78, 94, 102, 191, 67, 184, 172, 122, 90]
 [RL 5/8] Step 220/300 | Reward=-0.0316 | Acc=0.2591 | AvgBudget=118.5 | StdBudget=40.6

Recent budgets: [184, 100, 113, 91, 91, 119, 82, 99, 198, 78]
 [RL 5/8] Step 240/300 | Reward=-0.0215 | Acc=0.2667 | AvgBudget=118.3 | StdBudget=40.4

Recent budgets: [157, 109, 102, 144, 208, 71, 82, 106, 67, 181]
 [RL 5/8] Step 260/300 | Reward=-0.0220 | Acc=0.2654 | AvgBudget=117.9 | StdBudget=39.6

Recent budgets: [89, 113, 92, 147, 114, 69, 140, 144, 141, 119]
 [RL 5/8] Step 280/300 | Reward=-0.0344 | Acc=0.2536 | AvgBudget=117.1 | StdBudget=38.9

Recent budgets: [111, 102, 154, 151, 120, 72, 103, 84, 110, 89]
 [RL 5/8] Step 300/300 | Reward=-0.0311 | Acc=0.2567 | AvgBudget=117.3 | StdBudget=39.2

Recent budgets: [69, 211, 66, 135, 94, 108, 105, 106, 112, 130]

=====

RL Epoch 5 summary

Train Accuracy : 0.2567

Train Avg Tokens: 117.3

Train Avg Reward: -0.0311

Train Avg Budget: 117.3 +/- 39.2

Deterministic policy evaluation on test set...

Det eval | 10/100 | Acc=0.4000 | AvgBudget=116.10 | StdBudget=9.93

Det eval | 20/100 | Acc=0.3000 | AvgBudget=116.50 | StdBudget=8.56

Det eval | 30/100 | Acc=0.3000 | AvgBudget=116.10 | StdBudget=8.13

Det eval | 40/100 | Acc=0.2750 | AvgBudget=114.60 | StdBudget=8.04

```

Det eval | 50/100 | Acc=0.2800 | AvgBudget=115.18 | StdBudget=8.13
Det eval | 60/100 | Acc=0.3000 | AvgBudget=115.12 | StdBudget=7.81
Det eval | 70/100 | Acc=0.3000 | AvgBudget=115.60 | StdBudget=7.67
Det eval | 80/100 | Acc=0.2875 | AvgBudget=115.86 | StdBudget=7.67
Det eval | 90/100 | Acc=0.3000 | AvgBudget=115.81 | StdBudget=7.53
Det eval | 100/100 | Acc=0.3000 | AvgBudget=115.73 | StdBudget=7.34
Test Accuracy : 0.3000
Test AvgTokens : 115.7
Test AvgReward : 0.0278
Test Budget : 115.7 +/- 7.3 [100, 134]
=====
=====

[RL 6/8] Step 20/300 | Reward=0.0847 | Acc=0.3500 | AvgBudget=117.6 |
StdBudget=30.4
Recent budgets: [99, 145, 126, 133, 92, 138, 142, 137, 183, 79]
[RL 6/8] Step 40/300 | Reward=0.0560 | Acc=0.3250 | AvgBudget=116.8 |
StdBudget=35.1
Recent budgets: [109, 90, 181, 199, 159, 167, 94, 74, 81, 162]
[RL 6/8] Step 60/300 | Reward=0.0130 | Acc=0.2833 | AvgBudget=113.7 |
StdBudget=32.7
Recent budgets: [101, 83, 150, 92, 157, 110, 141, 77, 110, 125]
[RL 6/8] Step 80/300 | Reward=-0.0275 | Acc=0.2500 | AvgBudget=113.3 |
StdBudget=31.5
Recent budgets: [98, 74, 102, 74, 118, 73, 95, 110, 136, 107]
[RL 6/8] Step 100/300 | Reward=0.0320 | Acc=0.3000 | AvgBudget=114.3 |
StdBudget=34.2
Recent budgets: [111, 116, 210, 155, 94, 94, 68, 65, 189, 102]
[RL 6/8] Step 120/300 | Reward=-0.0012 | Acc=0.2833 | AvgBudget=118.5 |
StdBudget=37.9
Recent budgets: [133, 69, 91, 172, 137, 212, 200, 144, 145, 81]
[RL 6/8] Step 140/300 | Reward=0.0055 | Acc=0.2857 | AvgBudget=117.2 |
StdBudget=39.1
Recent budgets: [70, 84, 69, 141, 151, 71, 118, 87, 106, 69]
[RL 6/8] Step 160/300 | Reward=-0.0085 | Acc=0.2750 | AvgBudget=117.4 |
StdBudget=38.6
Recent budgets: [119, 104, 124, 73, 196, 92, 83, 135, 73, 155]
[RL 6/8] Step 180/300 | Reward=-0.0203 | Acc=0.2667 | AvgBudget=117.9 |
StdBudget=38.3
Recent budgets: [148, 133, 103, 82, 74, 155, 147, 86, 167, 75]
[RL 6/8] Step 200/300 | Reward=-0.0251 | Acc=0.2600 | AvgBudget=116.7 |
StdBudget=38.2
Recent budgets: [114, 98, 126, 78, 82, 178, 67, 198, 95, 68]
[RL 6/8] Step 220/300 | Reward=-0.0076 | Acc=0.2727 | AvgBudget=116.2 |
StdBudget=37.8
Recent budgets: [82, 108, 108, 85, 68, 77, 133, 117, 140, 192]
[RL 6/8] Step 240/300 | Reward=-0.0212 | Acc=0.2625 | AvgBudget=116.4 |
StdBudget=37.0

```

Recent budgets: [123, 153, 115, 158, 160, 128, 75, 73, 107, 99]
[RL 6/8] Step 260/300 | Reward=-0.0308 | Acc=0.2577 | AvgBudget=117.6 | StdBudget=37.6

Recent budgets: [84, 209, 79, 176, 150, 95, 192, 182, 130, 80]
[RL 6/8] Step 280/300 | Reward=-0.0307 | Acc=0.2571 | AvgBudget=117.4 | StdBudget=37.8

Recent budgets: [70, 103, 85, 205, 120, 131, 106, 169, 158, 86]
[RL 6/8] Step 300/300 | Reward=-0.0346 | Acc=0.2567 | AvgBudget=118.5 | StdBudget=38.1

Recent budgets: [96, 127, 123, 193, 192, 177, 134, 132, 106, 141]

=====

RL Epoch 6 summary

Train Accuracy : 0.2567

Train Avg Tokens: 118.5

Train Avg Reward: -0.0346

Train Avg Budget: 118.5 +/- 38.1

Deterministic policy evaluation on test set...

Det eval | 10/100 | Acc=0.3000 | AvgBudget=118.10 | StdBudget=9.62

Det eval | 20/100 | Acc=0.2500 | AvgBudget=118.60 | StdBudget=8.18

Det eval | 30/100 | Acc=0.2667 | AvgBudget=118.17 | StdBudget=8.01

Det eval | 40/100 | Acc=0.2500 | AvgBudget=116.58 | StdBudget=8.02

Det eval | 50/100 | Acc=0.2800 | AvgBudget=117.18 | StdBudget=8.01

Det eval | 60/100 | Acc=0.3000 | AvgBudget=117.05 | StdBudget=7.73

Det eval | 70/100 | Acc=0.3143 | AvgBudget=117.54 | StdBudget=7.59

Det eval | 80/100 | Acc=0.3000 | AvgBudget=117.83 | StdBudget=7.56

Det eval | 90/100 | Acc=0.3000 | AvgBudget=117.72 | StdBudget=7.44

Det eval | 100/100 | Acc=0.3000 | AvgBudget=117.64 | StdBudget=7.25

Test Accuracy : 0.3000

Test AvgTokens : 117.6

Test AvgReward : 0.0221

Test Budget : 117.6 +/- 7.2 [102, 135]

=====

[RL 7/8] Step 20/300 | Reward=-0.1839 | Acc=0.1500 | AvgBudget=123.8 | StdBudget=30.2

Recent budgets: [139, 133, 107, 105, 104, 119, 119, 155, 150, 119]

[RL 7/8] Step 40/300 | Reward=-0.0997 | Acc=0.2000 | AvgBudget=116.5 | StdBudget=31.0

Recent budgets: [76, 71, 69, 121, 154, 100, 135, 96, 108, 143]

[RL 7/8] Step 60/300 | Reward=-0.0770 | Acc=0.2167 | AvgBudget=116.0 | StdBudget=32.8

Recent budgets: [167, 134, 95, 99, 94, 135, 150, 131, 201, 97]

[RL 7/8] Step 80/300 | Reward=-0.0451 | Acc=0.2375 | AvgBudget=114.0 | StdBudget=34.7

Recent budgets: [109, 98, 69, 114, 106, 107, 103, 137, 69, 68]
[RL 7/8] Step 100/300 | Reward=-0.0629 | Acc=0.2300 | AvgBudget=116.8 | StdBudget=36.5

Recent budgets: [170, 203, 99, 154, 197, 170, 151, 70, 106, 70]
[RL 7/8] Step 120/300 | Reward=-0.0573 | Acc=0.2333 | AvgBudget=116.3 | StdBudget=36.3

Recent budgets: [153, 78, 97, 99, 114, 139, 123, 162, 75, 74]
[RL 7/8] Step 140/300 | Reward=-0.0444 | Acc=0.2357 | AvgBudget=113.0 | StdBudget=36.2

Recent budgets: [70, 138, 72, 95, 66, 74, 171, 68, 102, 75]
[RL 7/8] Step 160/300 | Reward=-0.0319 | Acc=0.2437 | AvgBudget=112.2 | StdBudget=35.4

Recent budgets: [115, 107, 139, 70, 119, 141, 91, 91, 113, 98]
[RL 7/8] Step 180/300 | Reward=-0.0401 | Acc=0.2444 | AvgBudget=115.2 | StdBudget=36.8

Recent budgets: [147, 117, 119, 131, 81, 94, 199, 122, 197, 191]
[RL 7/8] Step 200/300 | Reward=-0.0346 | Acc=0.2500 | AvgBudget=115.7 | StdBudget=37.2

Recent budgets: [146, 69, 175, 102, 81, 172, 132, 151, 104, 81]
[RL 7/8] Step 220/300 | Reward=-0.0350 | Acc=0.2500 | AvgBudget=115.8 | StdBudget=37.8

Recent budgets: [127, 212, 122, 67, 89, 78, 80, 108, 158, 92]
[RL 7/8] Step 240/300 | Reward=-0.0359 | Acc=0.2500 | AvgBudget=116.1 | StdBudget=37.6

Recent budgets: [202, 153, 67, 69, 106, 72, 125, 136, 82, 131]
[RL 7/8] Step 260/300 | Reward=-0.0253 | Acc=0.2577 | AvgBudget=115.8 | StdBudget=37.7

Recent budgets: [66, 79, 76, 81, 154, 90, 132, 68, 84, 165]
[RL 7/8] Step 280/300 | Reward=-0.0195 | Acc=0.2607 | AvgBudget=115.1 | StdBudget=37.4

Recent budgets: [135, 114, 89, 120, 76, 117, 83, 132, 118, 68]
[RL 7/8] Step 300/300 | Reward=-0.0226 | Acc=0.2600 | AvgBudget=115.9 | StdBudget=37.6

Recent budgets: [158, 137, 199, 119, 74, 68, 137, 145, 72, 124]

=====

RL Epoch 7 summary

Train Accuracy : 0.2600

Train Avg Tokens: 115.9

Train Avg Reward: -0.0226

Train Avg Budget: 115.9 +/- 37.6

Deterministic policy evaluation on test set...

Det eval | 10/100 | Acc=0.4000 | AvgBudget=112.80 | StdBudget=9.84

Det eval | 20/100 | Acc=0.3000 | AvgBudget=113.10 | StdBudget=8.38

Det eval | 30/100 | Acc=0.3000 | AvgBudget=112.80 | StdBudget=8.31

Det eval | 40/100 | Acc=0.3250 | AvgBudget=111.30 | StdBudget=8.20

```

Det eval | 50/100 | Acc=0.3400 | AvgBudget=111.92 | StdBudget=8.28
Det eval | 60/100 | Acc=0.3500 | AvgBudget=111.72 | StdBudget=8.01
Det eval | 70/100 | Acc=0.3429 | AvgBudget=112.19 | StdBudget=7.83
Det eval | 80/100 | Acc=0.3375 | AvgBudget=112.42 | StdBudget=7.84
Det eval | 90/100 | Acc=0.3333 | AvgBudget=112.33 | StdBudget=7.80
Det eval | 100/100 | Acc=0.3200 | AvgBudget=112.21 | StdBudget=7.60
Test Accuracy : 0.3200
Test AvgTokens : 112.2
Test AvgReward : 0.0634
Test Budget : 112.2 +/- 7.6 [96, 131]
=====
=====

[RL 8/8] Step 20/300 | Reward=0.1477 | Acc=0.3500 | AvgBudget=96.6 |
StdBudget=36.2
Recent budgets: [83, 188, 81, 66, 68, 68, 81, 168, 72, 88]
[RL 8/8] Step 40/300 | Reward=0.0536 | Acc=0.3000 | AvgBudget=107.2 |
StdBudget=38.2
Recent budgets: [145, 187, 176, 158, 114, 124, 169, 65, 85, 78]
[RL 8/8] Step 60/300 | Reward=0.1198 | Acc=0.3667 | AvgBudget=112.8 |
StdBudget=40.8
Recent budgets: [204, 149, 153, 109, 137, 180, 154, 70, 178, 129]
[RL 8/8] Step 80/300 | Reward=0.0891 | Acc=0.3625 | AvgBudget=121.3 |
StdBudget=44.9
Recent budgets: [104, 198, 96, 178, 68, 143, 160, 186, 84, 174]
[RL 8/8] Step 100/300 | Reward=0.0789 | Acc=0.3500 | AvgBudget=119.5 |
StdBudget=43.7
Recent budgets: [85, 127, 76, 123, 137, 191, 81, 186, 106, 107]
[RL 8/8] Step 120/300 | Reward=0.0604 | Acc=0.3333 | AvgBudget=118.8 |
StdBudget=41.8
Recent budgets: [96, 128, 171, 74, 83, 120, 136, 173, 97, 119]
[RL 8/8] Step 140/300 | Reward=0.0453 | Acc=0.3214 | AvgBudget=118.8 |
StdBudget=41.1
Recent budgets: [95, 144, 161, 92, 77, 76, 164, 128, 169, 157]
[RL 8/8] Step 160/300 | Reward=0.0454 | Acc=0.3187 | AvgBudget=117.7 |
StdBudget=40.5
Recent budgets: [173, 145, 85, 94, 83, 78, 193, 107, 125, 161]
[RL 8/8] Step 180/300 | Reward=0.0262 | Acc=0.3000 | AvgBudget=116.3 |
StdBudget=39.8
Recent budgets: [118, 71, 69, 147, 96, 141, 102, 107, 110, 145]
[RL 8/8] Step 200/300 | Reward=0.0145 | Acc=0.2900 | AvgBudget=116.0 |
StdBudget=39.3
Recent budgets: [108, 84, 109, 104, 156, 148, 78, 77, 160, 75]
[RL 8/8] Step 220/300 | Reward=0.0101 | Acc=0.2864 | AvgBudget=115.9 |
StdBudget=38.9
Recent budgets: [73, 119, 144, 150, 116, 125, 175, 154, 77, 101]
[RL 8/8] Step 240/300 | Reward=0.0132 | Acc=0.2917 | AvgBudget=117.1 |
StdBudget=39.1

```


Recent budgets: [137, 172, 135, 116, 207, 201, 139, 88, 72, 120]
[RL 8/8] Step 260/300 | Reward=-0.0106 | Acc=0.2769 | AvgBudget=118.9 | StdBudget=39.8

Recent budgets: [169, 198, 177, 187, 201, 102, 193, 181, 95, 102]
[RL 8/8] Step 280/300 | Reward=-0.0146 | Acc=0.2714 | AvgBudget=118.0 | StdBudget=39.7

Recent budgets: [146, 65, 75, 79, 116, 73, 69, 75, 79, 139]
[RL 8/8] Step 300/300 | Reward=-0.0042 | Acc=0.2833 | AvgBudget=119.4 | StdBudget=40.0

Recent budgets: [88, 178, 133, 95, 170, 192, 99, 141, 140, 174]

=====

RL Epoch 8 summary

Train Accuracy : 0.2833

Train Avg Tokens: 119.4

Train Avg Reward: -0.0042

Train Avg Budget: 119.4 +/- 40.0

Deterministic policy evaluation on test set...

Det eval	10/100	Acc=0.3000	AvgBudget=121.50	StdBudget=9.99
Det eval	20/100	Acc=0.2500	AvgBudget=121.90	StdBudget=7.91
Det eval	30/100	Acc=0.2667	AvgBudget=122.10	StdBudget=7.24
Det eval	40/100	Acc=0.2250	AvgBudget=120.88	StdBudget=7.28
Det eval	50/100	Acc=0.2600	AvgBudget=121.18	StdBudget=7.79
Det eval	60/100	Acc=0.2833	AvgBudget=121.00	StdBudget=7.63
Det eval	70/100	Acc=0.2857	AvgBudget=121.41	StdBudget=7.38
Det eval	80/100	Acc=0.2750	AvgBudget=121.72	StdBudget=7.48
Det eval	90/100	Acc=0.2778	AvgBudget=121.59	StdBudget=7.51
Det eval	100/100	Acc=0.2800	AvgBudget=121.44	StdBudget=7.36

Test Accuracy : 0.2800

Test AvgTokens : 121.4

Test AvgReward : -0.0143

Test Budget : 121.4 +/- 7.4 [105, 137]

=====

Loading best saved policy/value for final evaluation...

Final deterministic policy evaluation...

Det eval	10/100	Acc=0.4000	AvgBudget=130.20	StdBudget=8.77
Det eval	20/100	Acc=0.3000	AvgBudget=130.55	StdBudget=7.00
Det eval	30/100	Acc=0.3000	AvgBudget=130.80	StdBudget=6.10
Det eval	40/100	Acc=0.2500	AvgBudget=129.93	StdBudget=6.10
Det eval	50/100	Acc=0.2800	AvgBudget=130.08	StdBudget=6.50
Det eval	60/100	Acc=0.3000	AvgBudget=130.03	StdBudget=6.34
Det eval	70/100	Acc=0.3143	AvgBudget=130.41	StdBudget=6.13
Det eval	80/100	Acc=0.3000	AvgBudget=130.60	StdBudget=6.21

```

Det eval | 90/100 | Acc=0.2889 | AvgBudget=130.53 | StdBudget=6.16
Det eval | 100/100 | Acc=0.3200 | AvgBudget=130.45 | StdBudget=6.04
{
  "final_deterministic_eval": {
    "accuracy": 0.32,
    "avg_thinking_tokens": 130.45,
    "avg_reward": 0.008649999999999988,
    "avg_budget": 130.45,
    "min_budget": 117.0,
    "max_budget": 145.0,
    "std_budget": 6.035519861619213
  }
}

```

```

Final sampled policy evaluation...
Samp eval | 10/100 | Acc=0.3000 | AvgBudget=125.90
Samp eval | 20/100 | Acc=0.2500 | AvgBudget=131.65
Samp eval | 30/100 | Acc=0.3333 | AvgBudget=142.23
Samp eval | 40/100 | Acc=0.3500 | AvgBudget=140.90
Samp eval | 50/100 | Acc=0.3200 | AvgBudget=134.44
Samp eval | 60/100 | Acc=0.3333 | AvgBudget=134.62
Samp eval | 70/100 | Acc=0.3429 | AvgBudget=137.09
Samp eval | 80/100 | Acc=0.3625 | AvgBudget=136.72
Samp eval | 90/100 | Acc=0.3444 | AvgBudget=135.98
Samp eval | 100/100 | Acc=0.3400 | AvgBudget=133.36
{
  "final_sampled_eval": {
    "accuracy": 0.34,
    "avg_thinking_tokens": 133.36,
    "avg_reward": 0.024919999999999987,
    "avg_budget": 133.36,
    "min_budget": 67.0,
    "max_budget": 215.0,
    "std_budget": 45.183740438347954
  }
}

```

```

Saved final policy to two_stage_policy_v10_final.pt
Saved final value net to two_stage_value_net_v10_final.pt

```

===== FINAL RESULTS COMPARISON =====

Method	Acc	AvgTokens	AvgReward
Fixed budget 96	0.2600	96	0.0370
Fixed budget 128	0.3300	128	0.0285
Fixed budget 144	0.3700	144	0.0305

Fixed budget 160	0.4900	160	0.1325
Fixed budget 176	0.5300	176	0.1345
Fixed budget 192	0.5200	192	0.0740
Fixed budget 220	0.5200	220	-0.0100
Policy deterministic	0.3200	130	0.0086
Policy sampled	0.3400	133	0.0249

=====

[3]: *# Disclaimer: For coding, I do have taken the help of Claude*

[]: